

MARI Drone

Team:

Seth Iris Canonigo

Connor Prior

Christopher Ruiz-Guerra

Nahom Solomon

Modular Aerial Real-Time Instrument (M.A.R.I.)

Flying Quadcopter Drone

Features:

- User control with Radio Controller
- Self-Stabilization
- Altitude-lock
- Modular - easily able to add additional features



Quadcopter Drone Flight

- We use 4 motors/propellers to control the drone
- Forward/back/left/right movement is achieved by tilting the drone using unequal levels of thrust in each motor
- Propellers must move in opposite directions so the drone doesn't spin in the same direction as the propellers
- Control Equations:

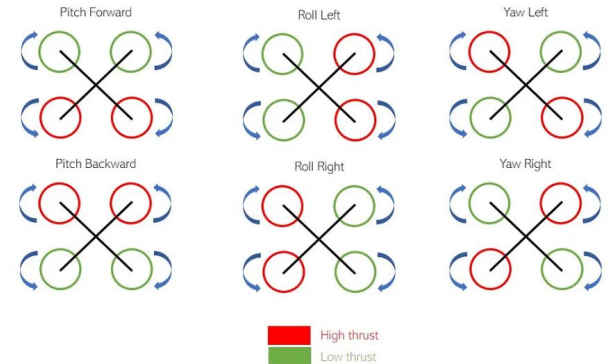
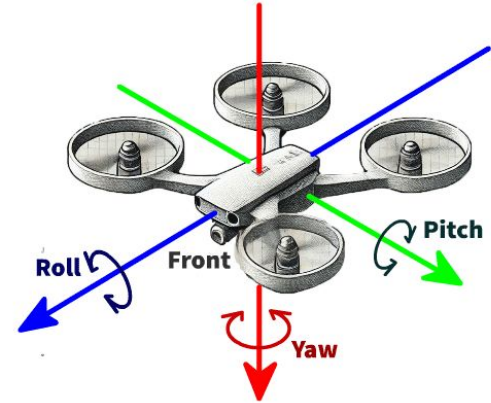
Motor(Front Left) = throttle - roll - pitch + yaw

Motor(Front Right) = throttle + roll - pitch - yaw

Motor(Rear Right) = throttle + roll + pitch + yaw

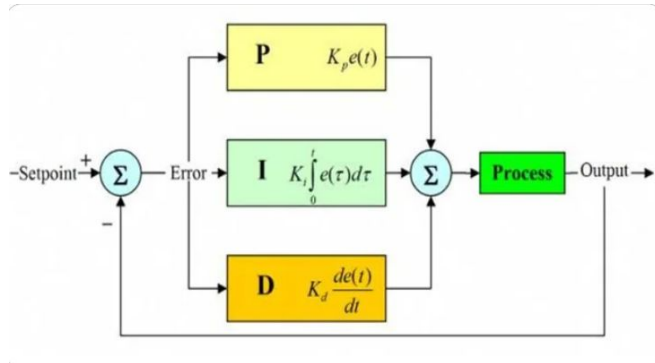
Motor(Rear Left) = throttle - roll + pitch - yaw

- Values for throttle, roll, pitch, and yaw will be controlled by user



Control System: PID

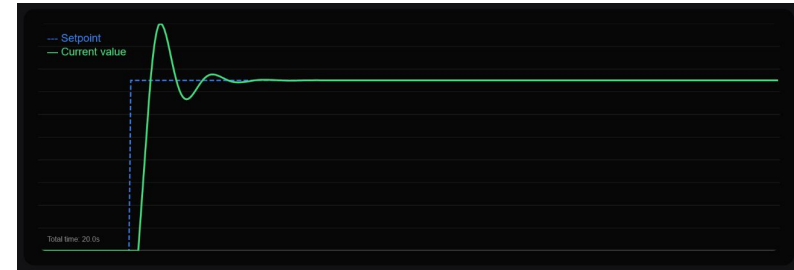
- Stands for Proportional Integral Derivative
 - **Proportional:** size of the error
 - **Integral:** area under curve, corrects for error over time
 - **Derivative:** rate of change of the error
- Used for reducing error because we want the output to our motors to stay as close to what the user intends as possible
- The sum of the P, I, and D components are needed for each of roll, pitch, and yaw



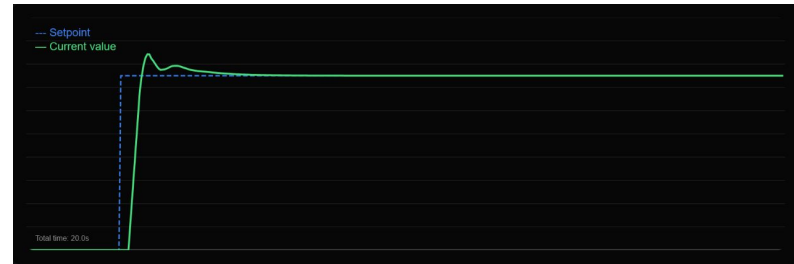
P



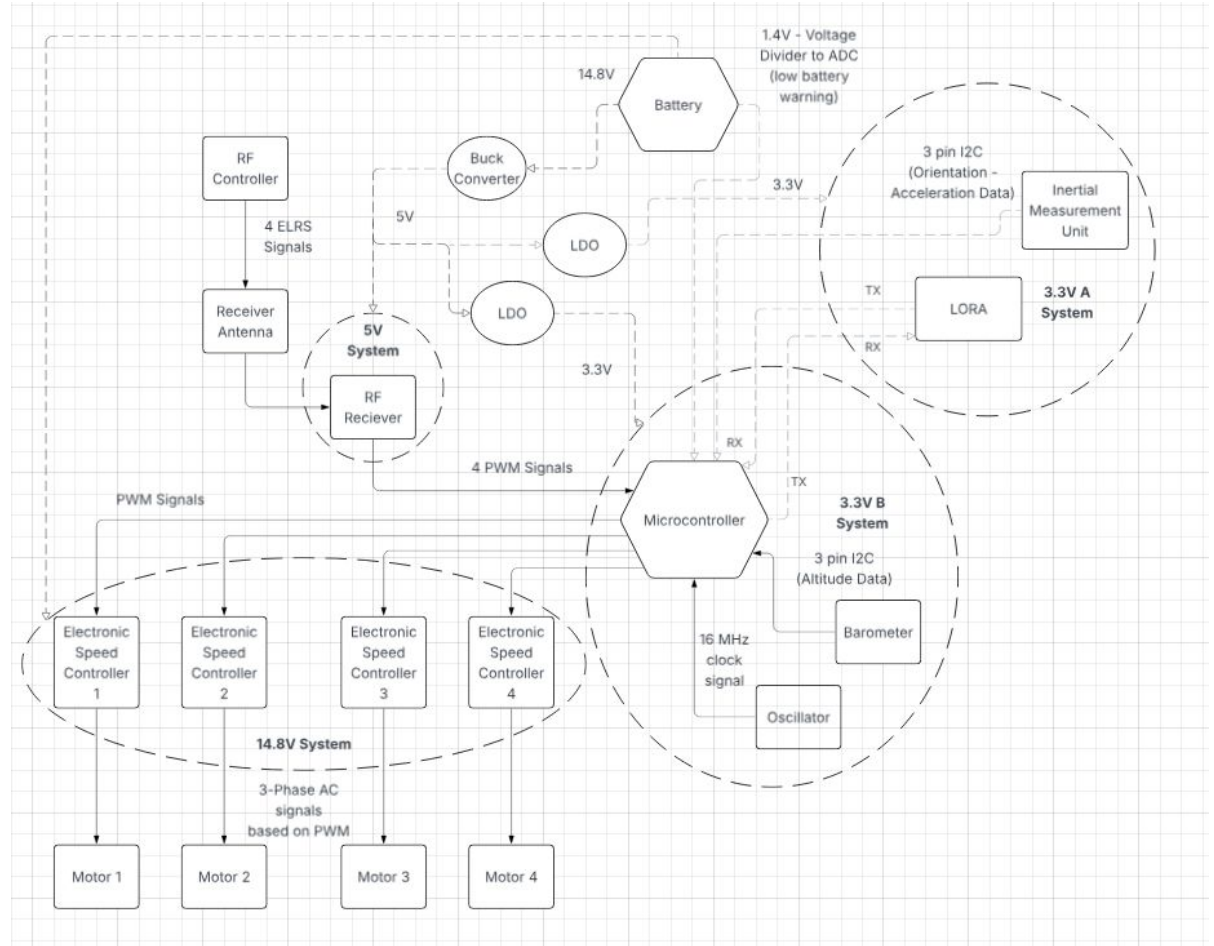
P+I



P+I+D



Block Diagram



Power

Key Priorities:

- Stable operation across battery voltage drop
- Noise mitigation and safe filtering
- Battery health monitoring safety



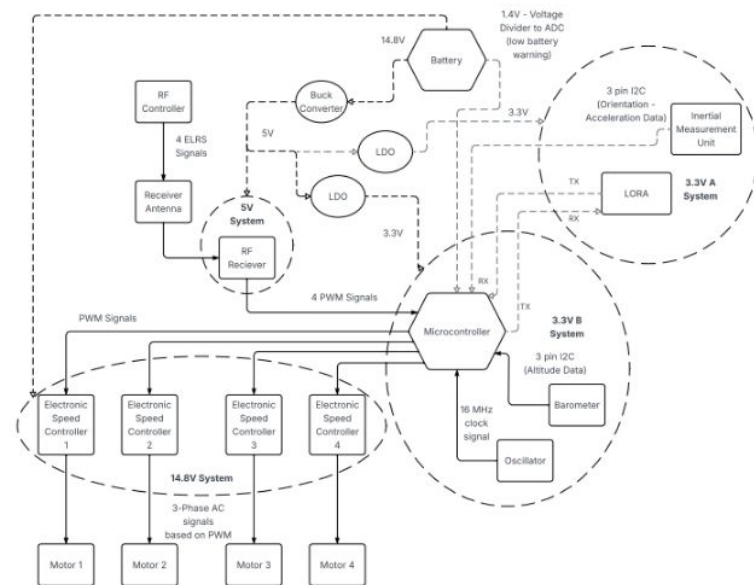
Power Overview

Power Architecture:

- **High Current Path:** Battery -> ESCs (up to 104 A / 16.8V and 110.4 A / 12.6 V)
- **Low Current Path:** Battery -> Buck Converter -> LDOs
- Separate regulated rails for MCU, RF Receiver, Sensors, LORA
- **Integrated ADC-based battery monitoring circuit** for live voltage tracking

Major Loads:

- ESCs: **104 A** Peak / 16.8 V (26 amps per ESC) or **110.4 A** Peak / 12.6 V (27.5 amps per ESC)
- 5 V Rail: **~0.84 A** (RF Receiver + LDO input)
- 3.3 V Rail A: **~0.203 A** (IMU, Lora)
- 3.3 V Rail B: **~0.409 A** (MCU, Barometer, Oscillator)



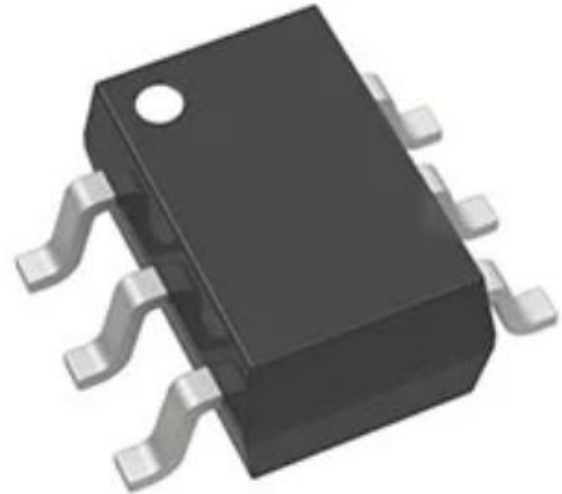
Battery: Gens Ace 3SP1 LiPo

- **11.1 V** nominal, **12.6 V** max
- **5300 mAh**, **60C** discharge
- **318 A** max continuous output capability
- Easily supports drone peak **~110 A** flight load
- XT90s main connector; supports high current draw
- Powers entire system
- Battery stress < 33% of rating
- Expected Flight Time (413g / 0.911 lbs):
 - Hover: 3.846 - 4.613 min
 - Ascend: 2.88 - 3.455 min



5 V Buck Converter - AP63300WU-7 (Diodes Inc.)

- 3A synchronous buck regulator
- Converts **16.8 V (max battery)** ---> **5 V regulated**
- High efficiency (~90%) to reduce heat and extend flight time
- Powers RF receiver and supplies input to two separate 3.3 V LDO rails
- LC output filter + bulk capacitance to minimize ripple from ESC switching

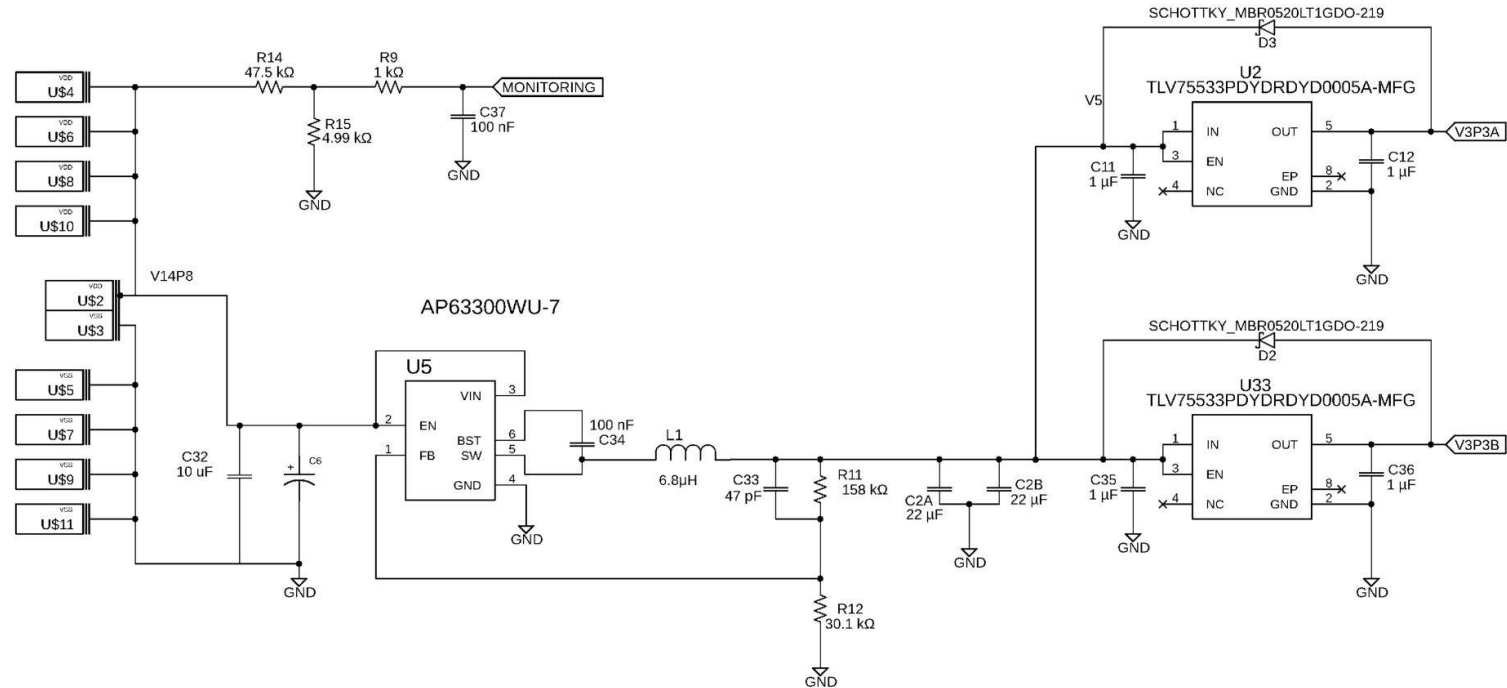


3.3 V LDO Regulator: TLV75533PDYDR (Texas Instruments)

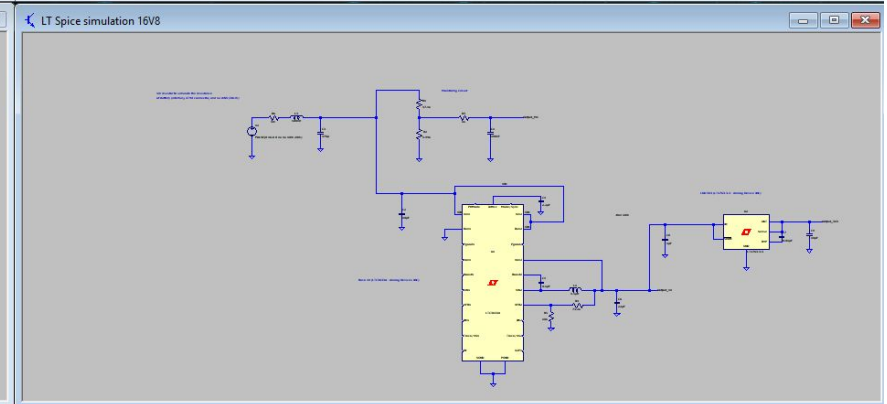
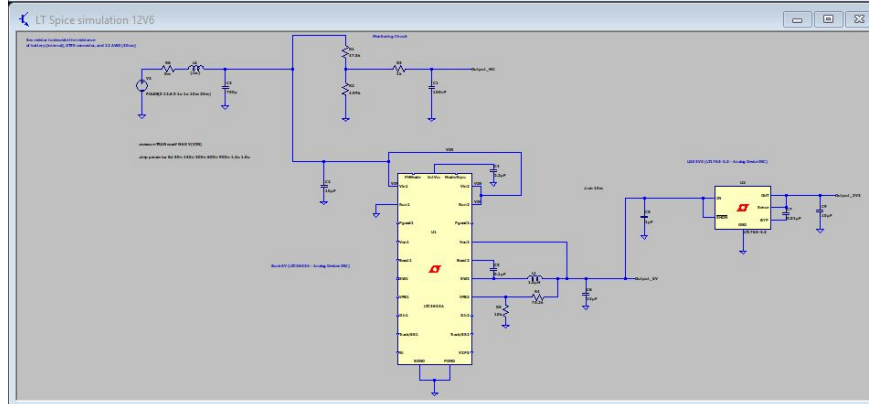
- Low noise, precision LDO rated for **500 mA** max
- Regulates 5 V → 3.3 V for MCU & sensors
- LDO A supports ~**204 mA** combined load (IMU, LORA)
- LDO B supports ~**409 mA** combined load (MCU, Barometer, Oscillator)
- Provides clean supply ideal for sensitive components



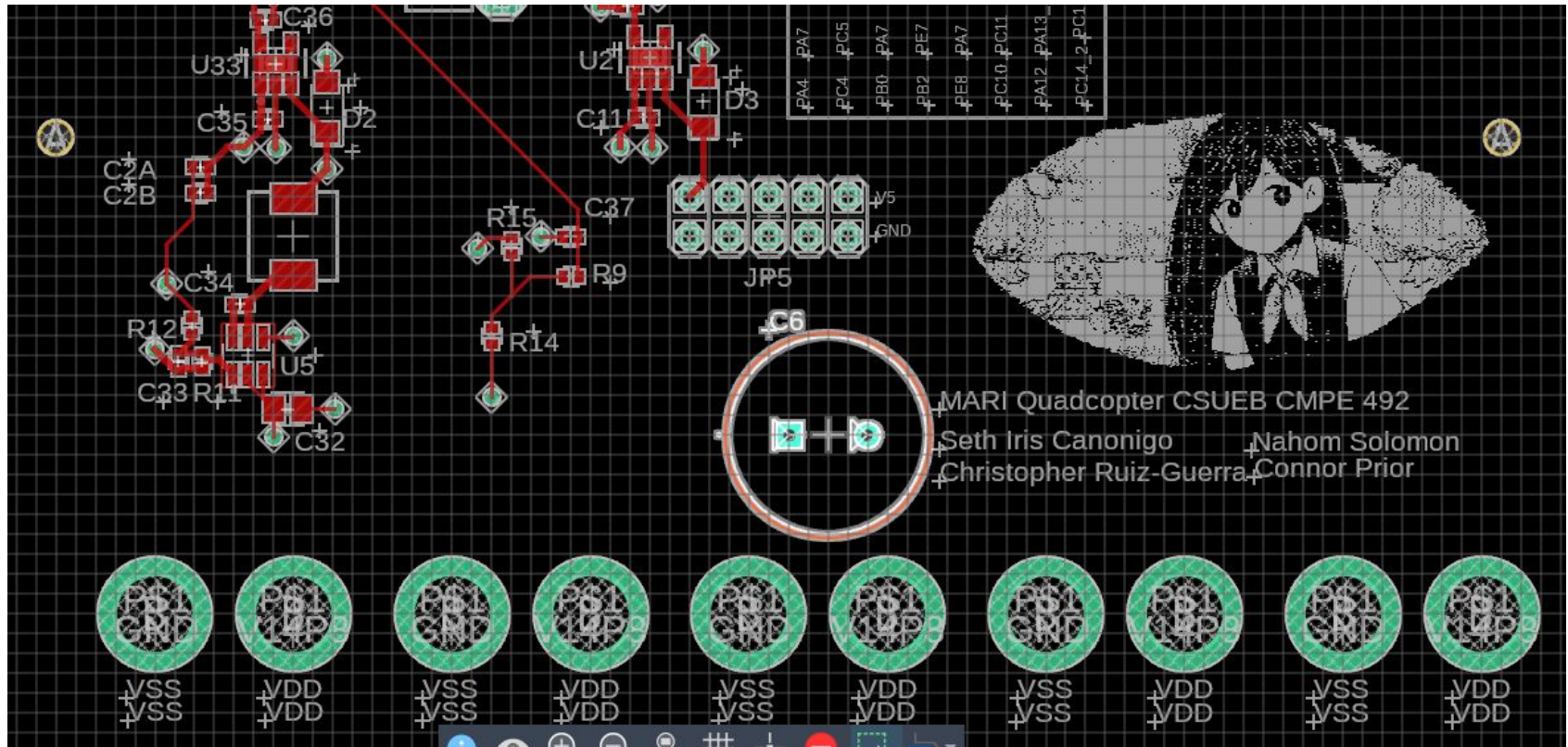
Schematic: Power



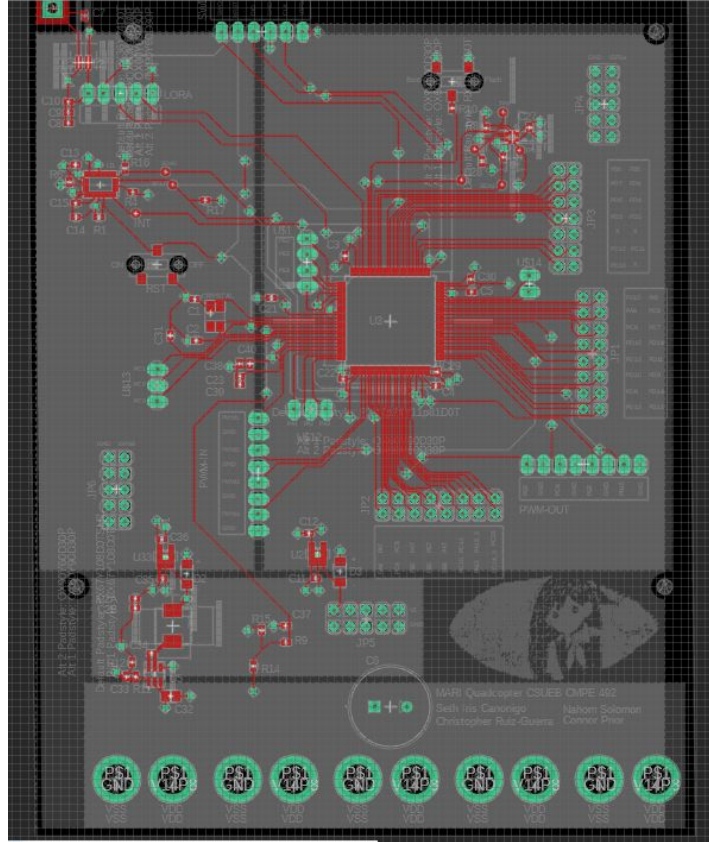
Schematic / LT Spice (12.6 V / 16.8 V)



PCB Layout: Power



PCB Layout: Power



Radio Controller / Receiver

Purpose

- Controller: Allows user to control the drone using control sticks.
Communicates with Radio Receiver in Express Long Range System (**ELRS**) protocol
- Receiver: Allows the use of 6 PWM Channels, Requiring only 4 Channels to control Pitch, Yaw, Roll and Throttle

Output

- Controller $\Rightarrow$ Receiver $\Rightarrow$ MCU $\Rightarrow$ ESC $\Rightarrow$ Motor
- PWM signals are taken as input for our MCU to calculate balanced PWM outputs to our ESC



Electronic Speed Controller(ESCs) : RCElectricParts 30A

- **Main Purpose:** Control rotational speed of motors
- Utilizes PWM signals to vary power from battery towards motors through MOSFET switching

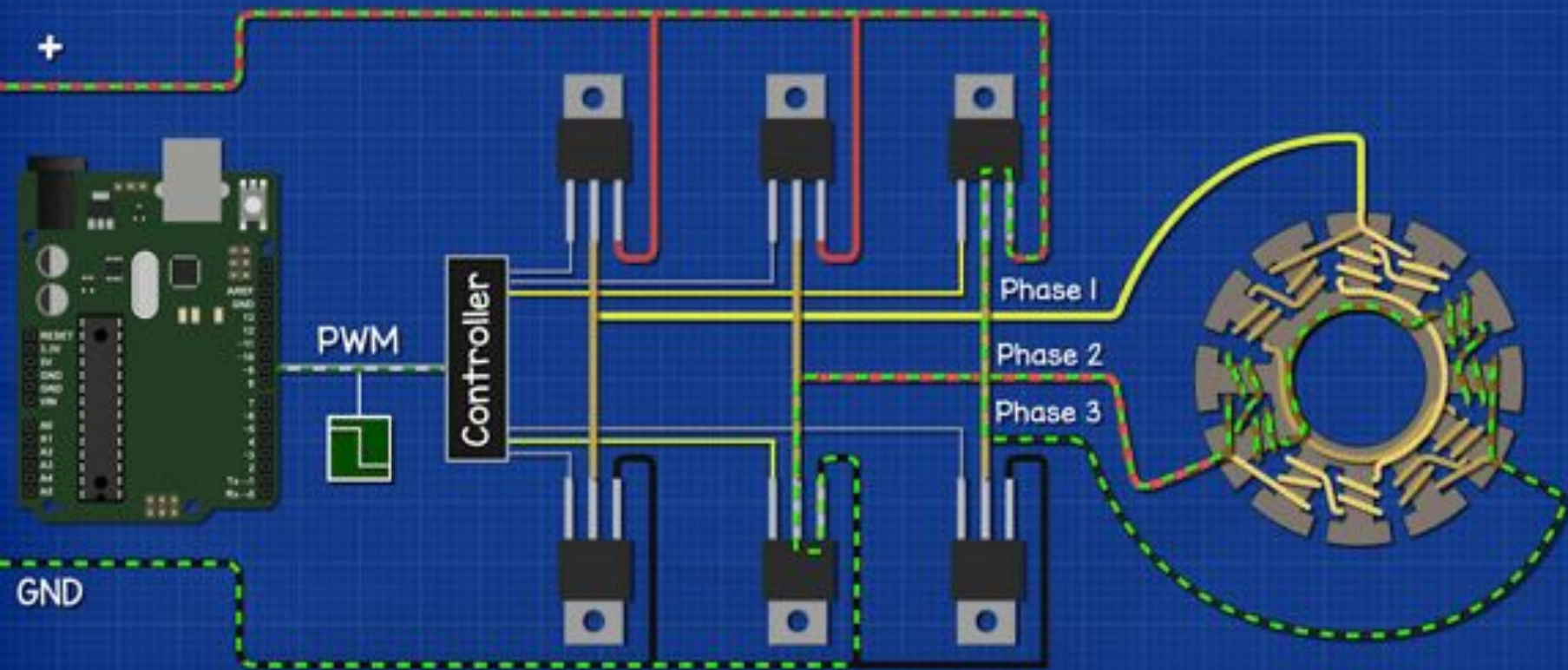


Motors: Emax ECO II Series 2807 3-6S 1300KV

- **Purpose:** Spins propellers to generate thrust per motor
 - Expected weight 5lbs
 - Each motor must produce 567 Grams of thrust
 - Allows maneuverability via flight equations

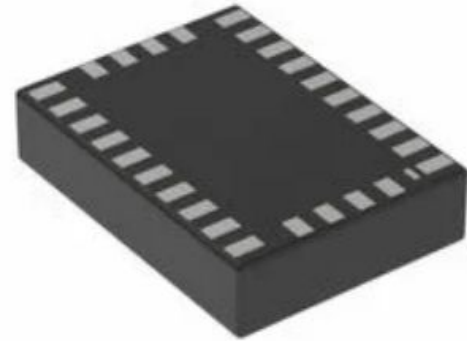


BLDC Motors Explained

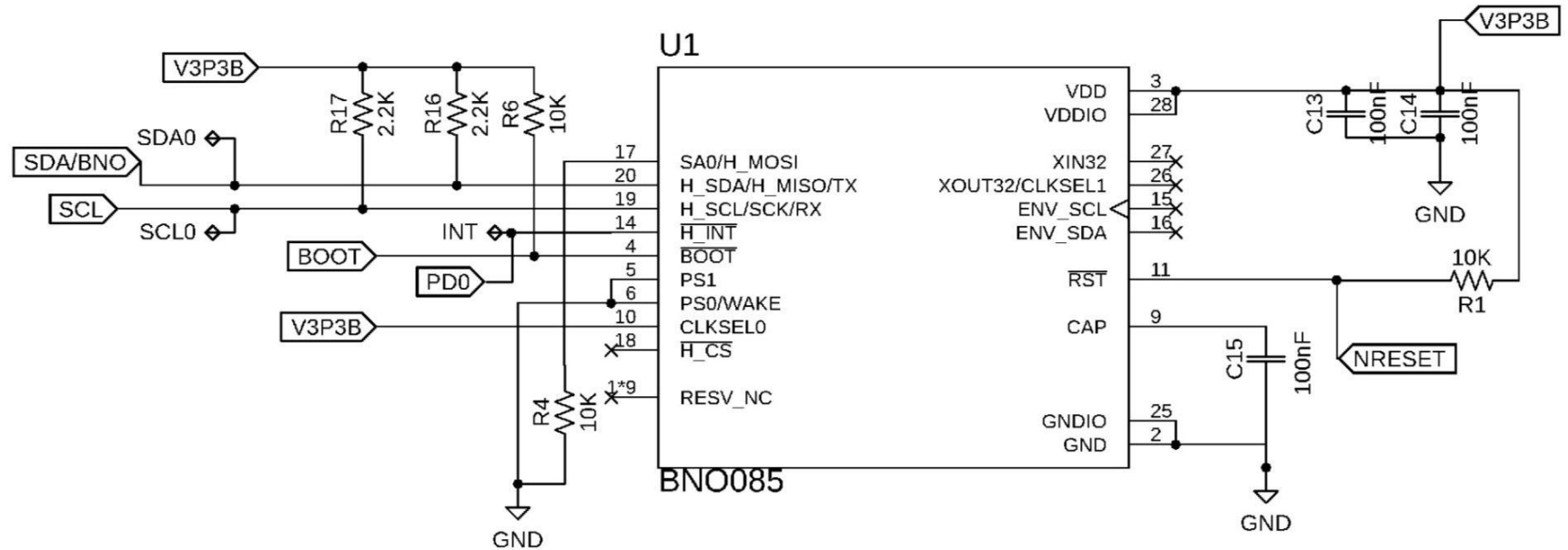


IMU (Inertial Measurement Unit): BNO085

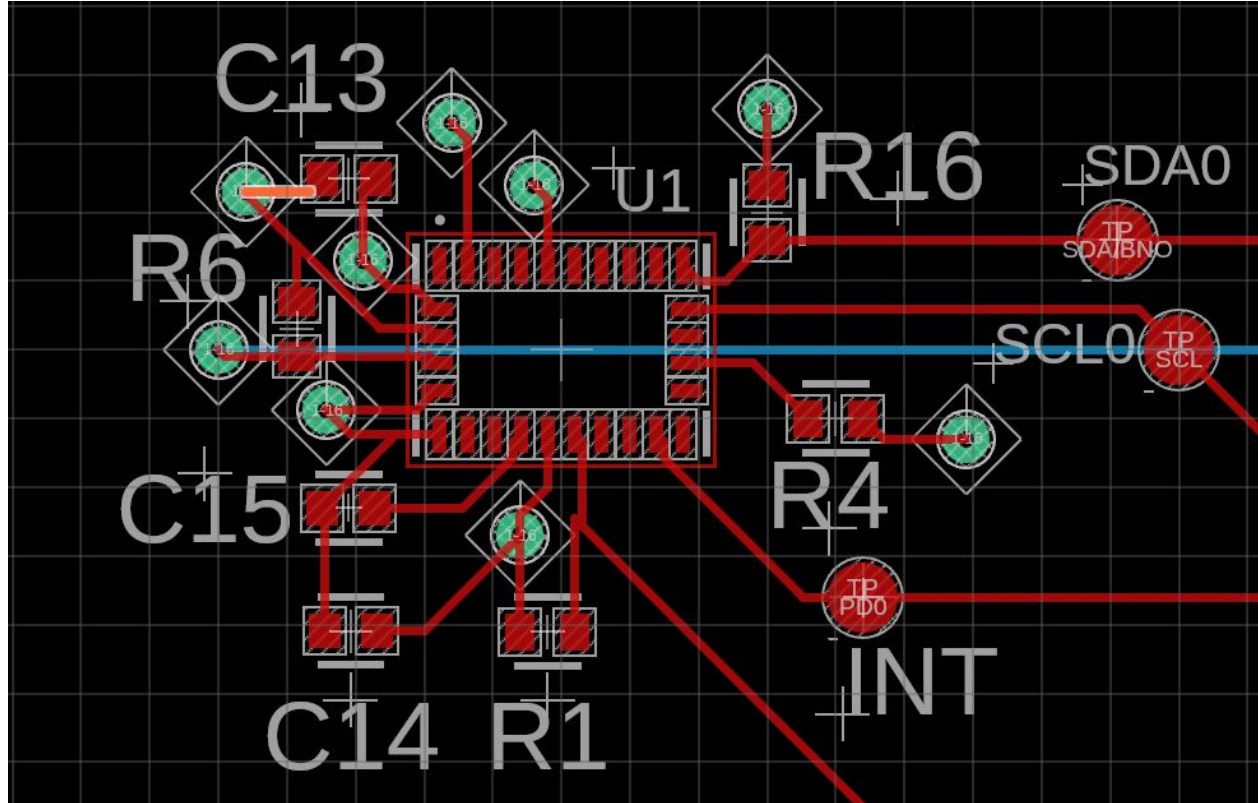
- **Main Purpose:** Stabilizing the Drone
- Detects the orientation and speed/acceleration of the drone
- Main Components:
 - Gyroscope - measures angular velocity
 - Accelerometer - measures linear acceleration
 - Magnetometer - measures direction of magnetic field
- Fusion mode allows the BNO085 to:
 - Calculate Euler Angles (Roll, Pitch, Yaw)
 - Minimize error using multiple sensors
- Sends data to the microcontroller through I2C protocol



Schematic: BNO085



PCB Layout: BNO085

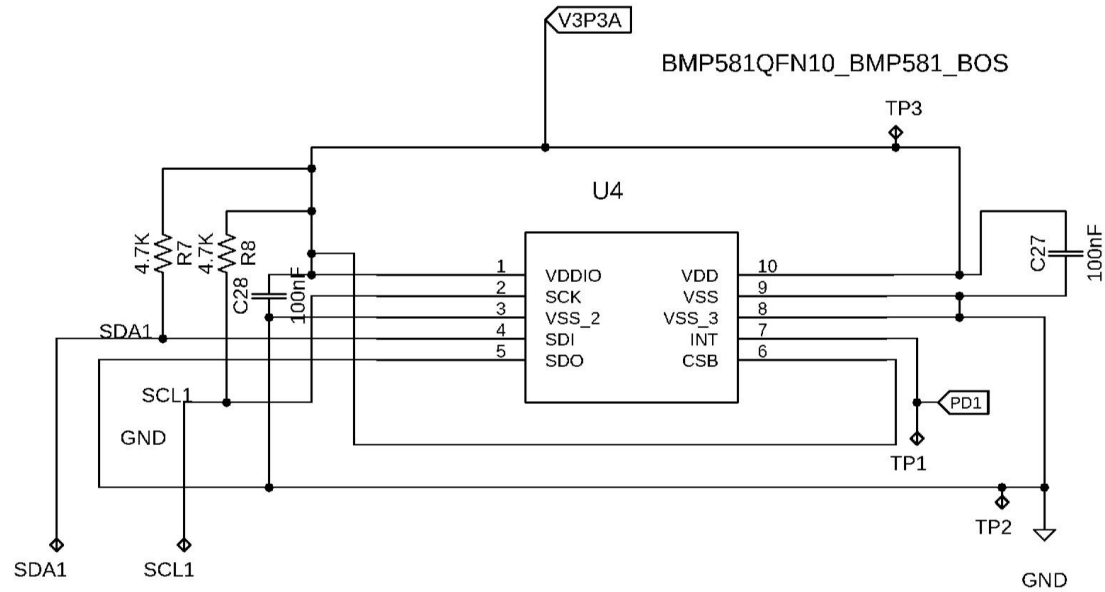


Barometer: BMP581

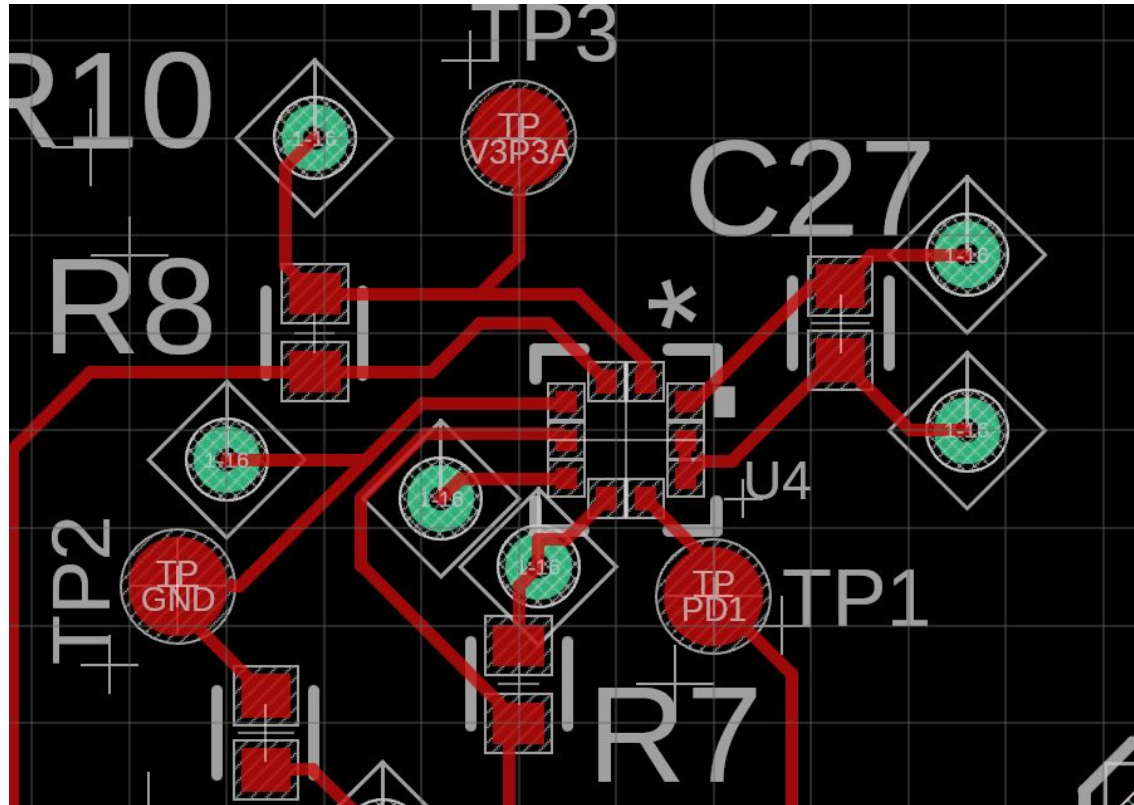
- Main purpose: measuring the altitude of the drone.
- Sends data to microcontroller through I2C.
- Data will be used to limit the height the drone will be able to fly at.



Schematic: BMP581



PCB Layout: BMP581



Microcontroller STM32F756VGT6

- MCU is the brain of MARI.
- Takes in input signals, runs all calculations, and outputs the PWM signals for the ESCs.
- Runs at 216 MHz with external crystal
- 100 total pins



IMU I2C (3 in signals)

13 input pins

Voltage warning light (1 in signal)

LoRa UART RX

5 out pins

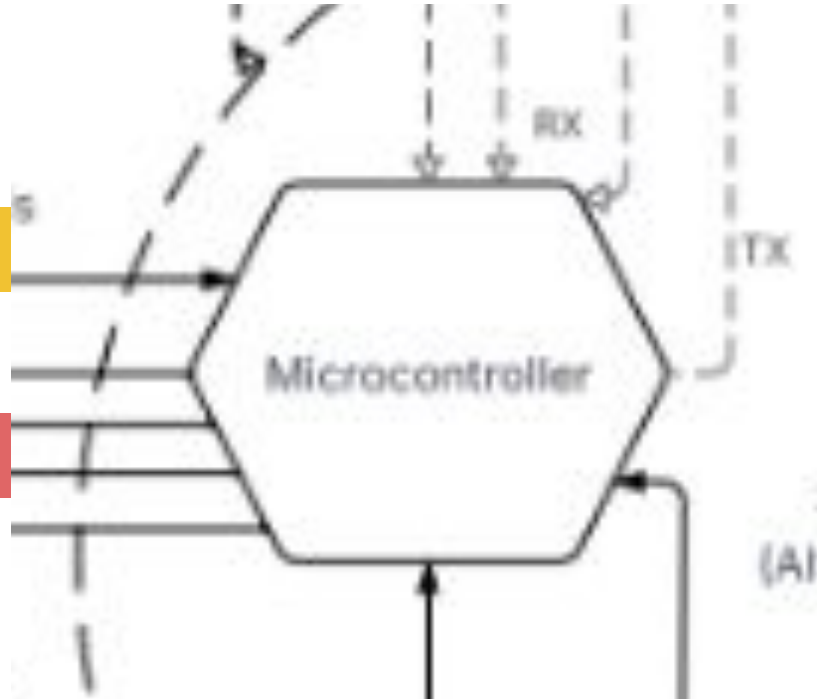
RF PWM (4 in signals)

LoRa UART TX

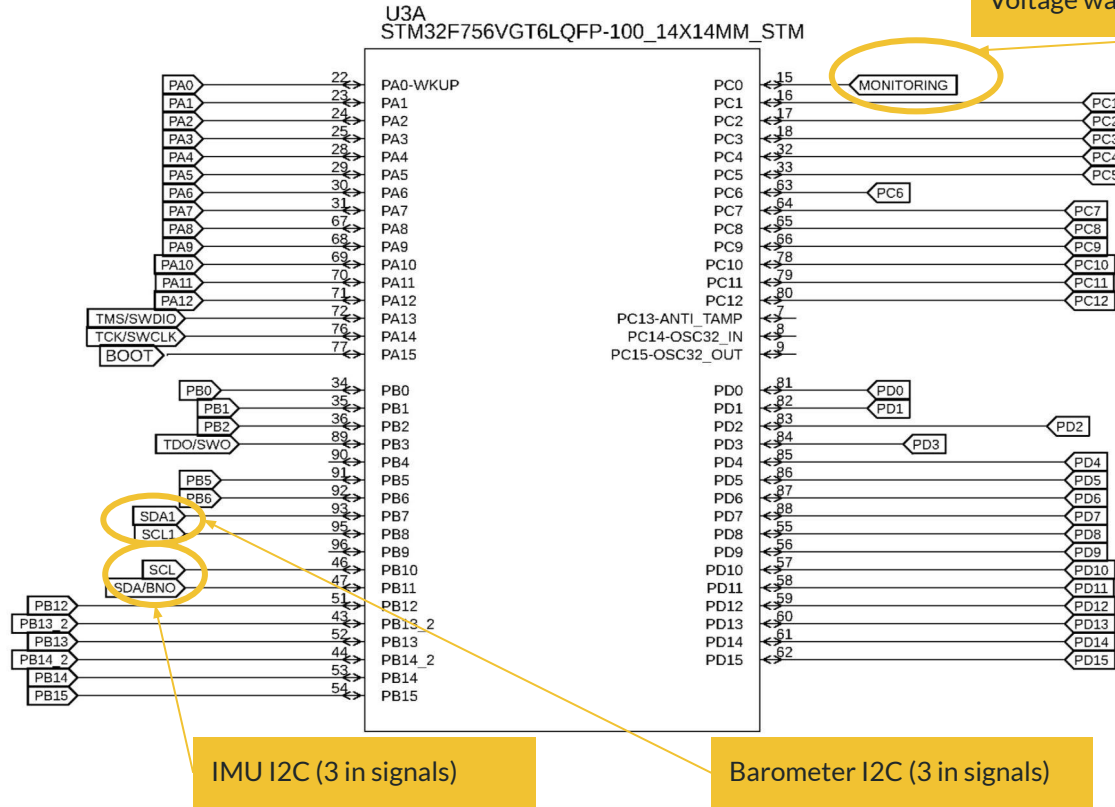
ESC PWM (4 out signals)

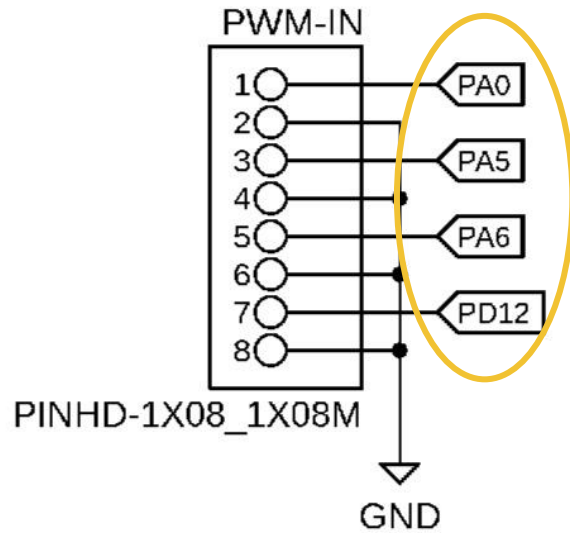
Barometer I2C (3 in signals)

16 MHz Oscillator (1 in signal)

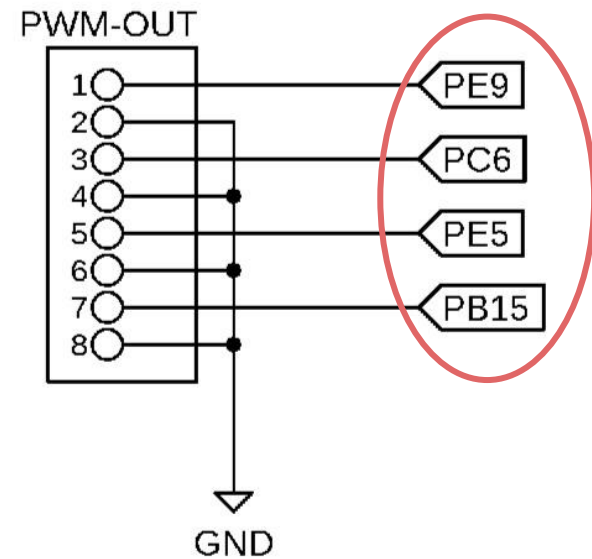


Schematic: Microcontroller 1



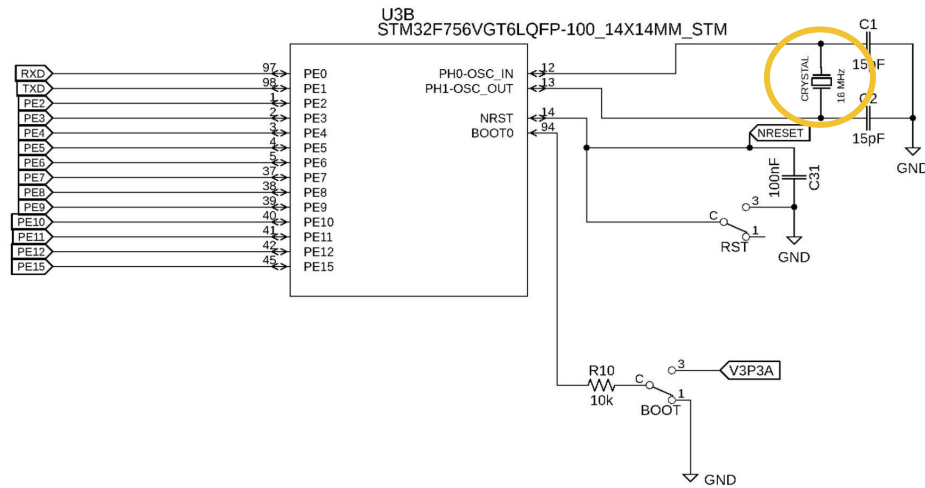


RF PWM (4 in signals)



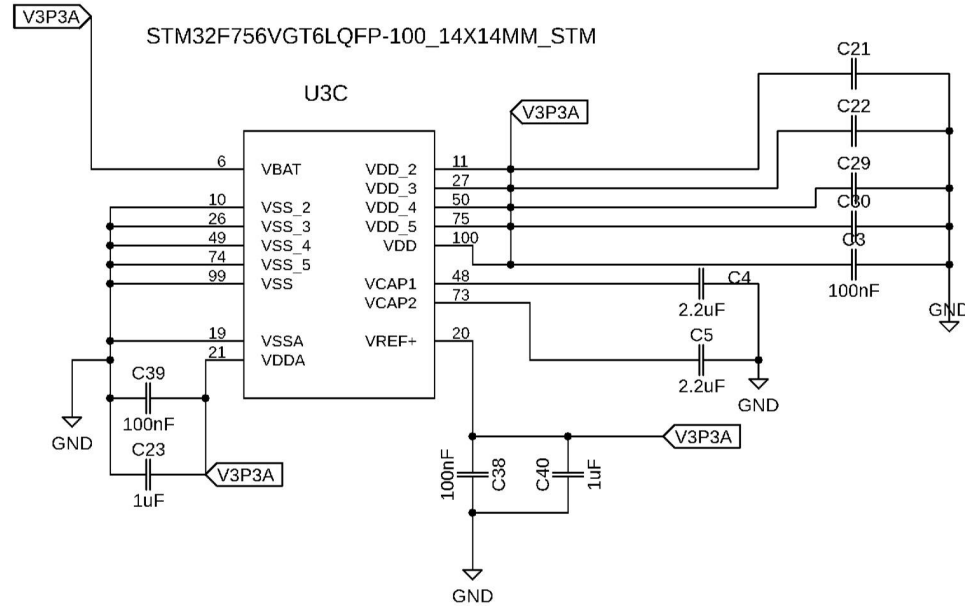
ESC PWM (4 out signals)

Schematic: Microcontroller 2

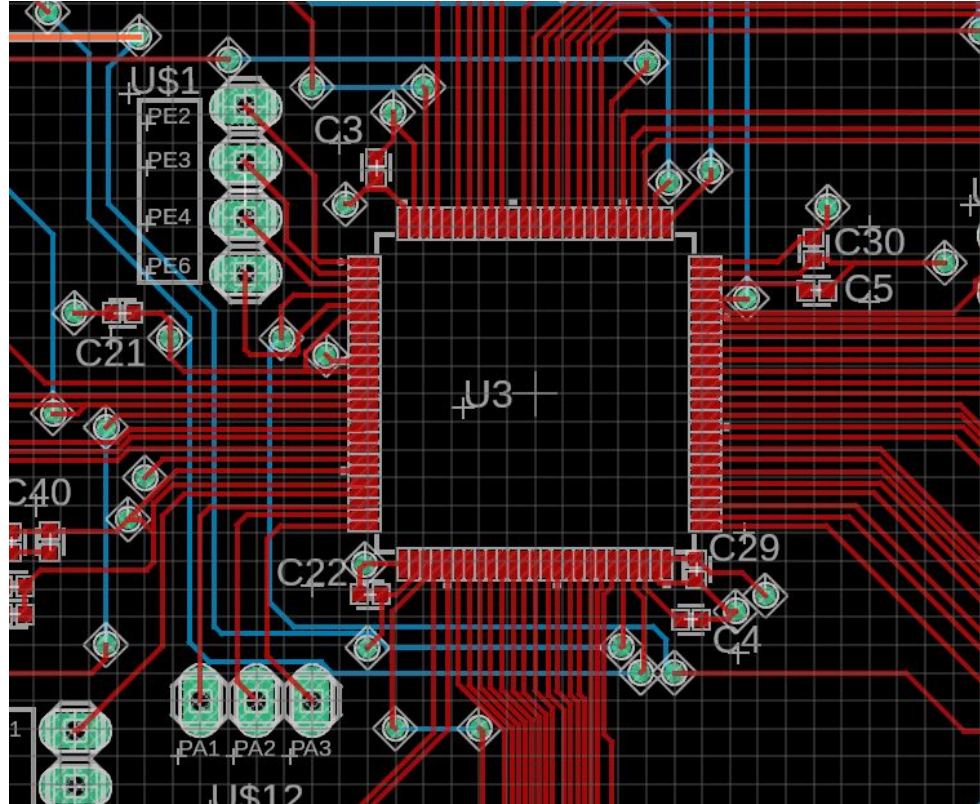


16 MHz Oscillator (1 in signal)

Schematic: Microcontroller 3



PCB Layout: Microcontroller



Long Range (LoRa) Telemetry: REYAX RYLR998

RYLR998 module is a UART-controlled LoRa transceiver.

It is low power (56.6 mA) and cheap

Can send data with AT+SEND=...

Can receive data with +RCV=...

Depending on configuration, it can go over a kilometer in distance with limited data rates.



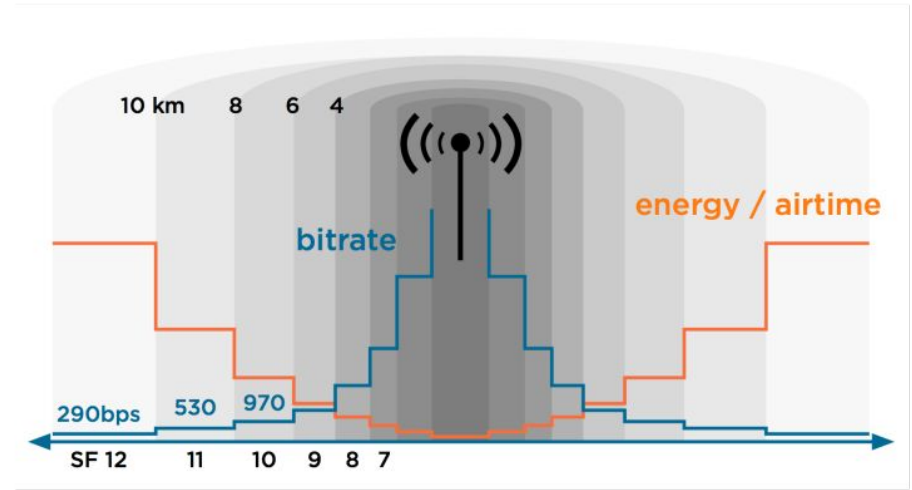
Hassles of LoRa

- Data rate is abysmal for real-time telemetry
- Only 240 Bytes per payload
- Default Throughput: 190 Bytes/s
- Default Latency 190.7 ms

If I send a packet per second, there will be data loss.

Obviously this is not including RX.

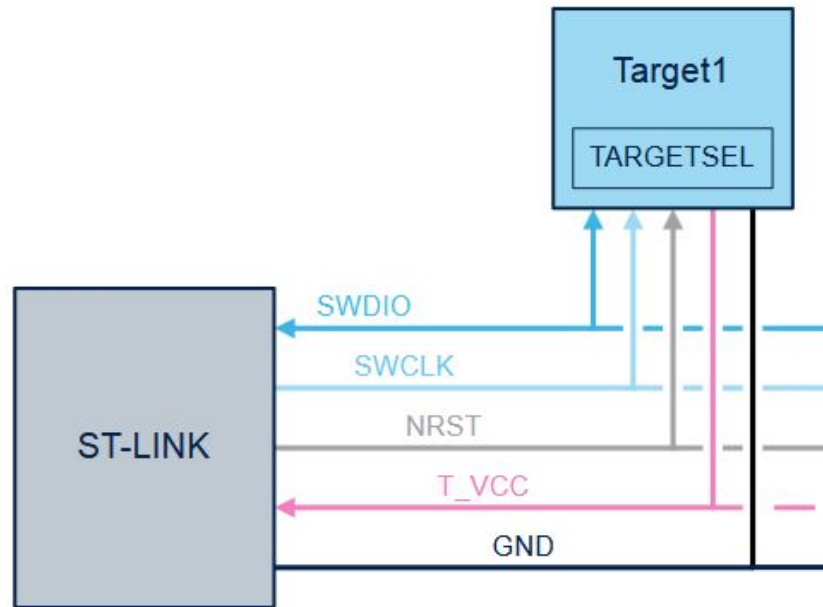
Meaning we had to be picky about what we send from the drone to the receiver



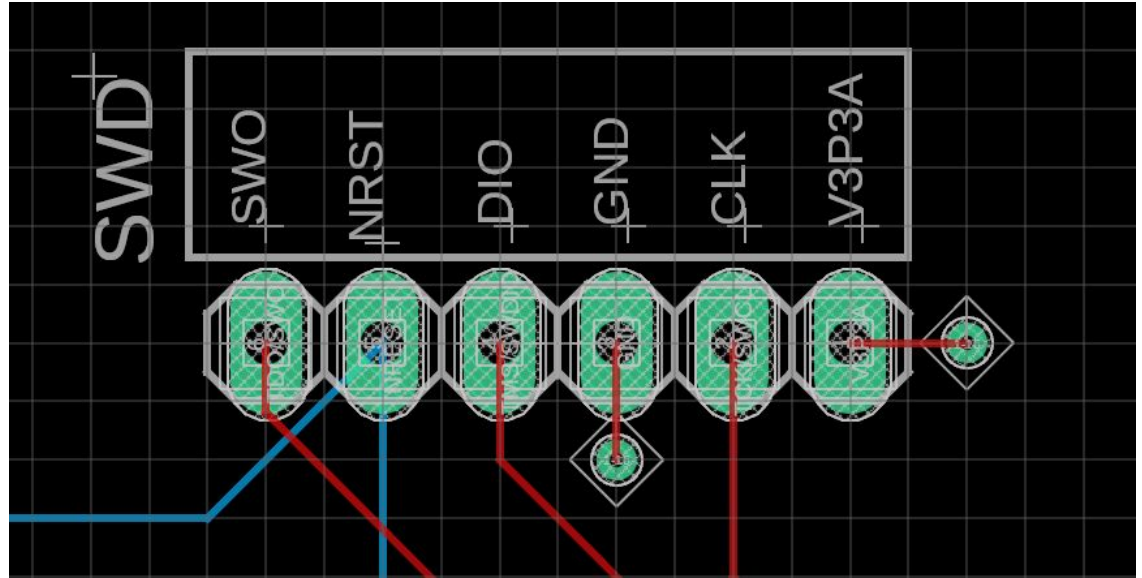
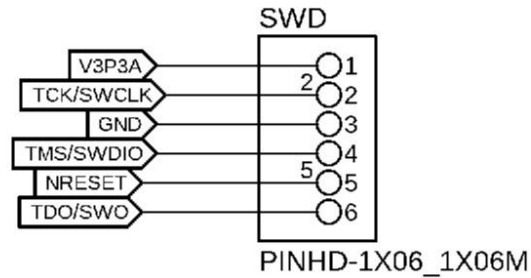
SWD (Serial Wire Debug) via ST-Link (SWJ)

Table 4. SWJ debug port pins

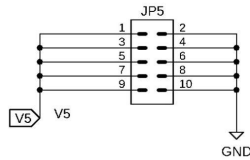
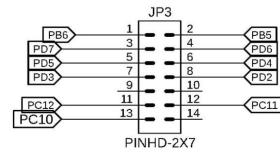
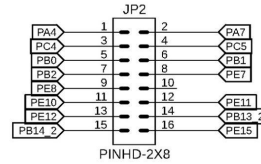
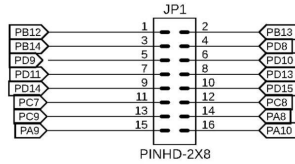
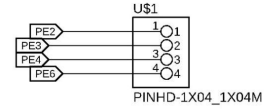
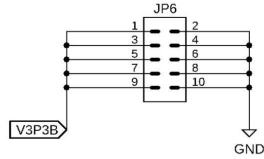
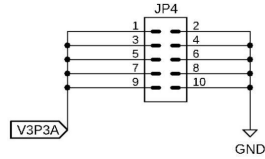
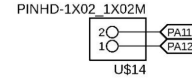
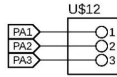
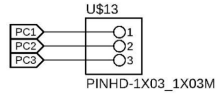
SWJ-DP pin name	JTAG debug port		SW debug port		Pin assignment
	Type	Description	Type	Debug assignment	
JTMS/SWDIO	I	JTAG test mode Selection	IO	Serial wire data input/output	PA13
JTCK/SWCLK	I	JTAG test clock	I	Serial wire clock	PA14
JTDI	I	JTAG test data input	-	-	PA15
JTDO/TRACESWO	O	JTAG test data output	-	TRACESWO if async trace is enabled	PB3
NJTRST	I	JTAG test nReset	-	-	PB4



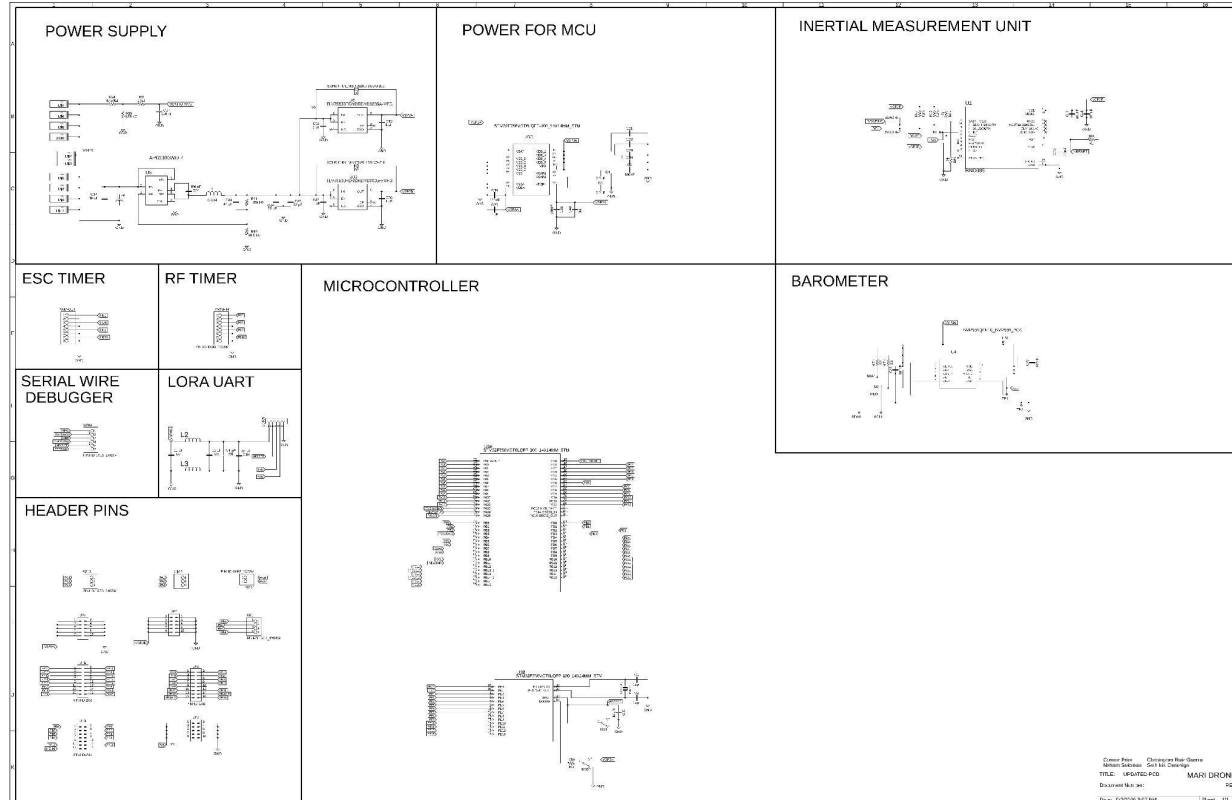
Schematic and PCB Layout: Serial Wire Debug Programming Pins



Extra Pins

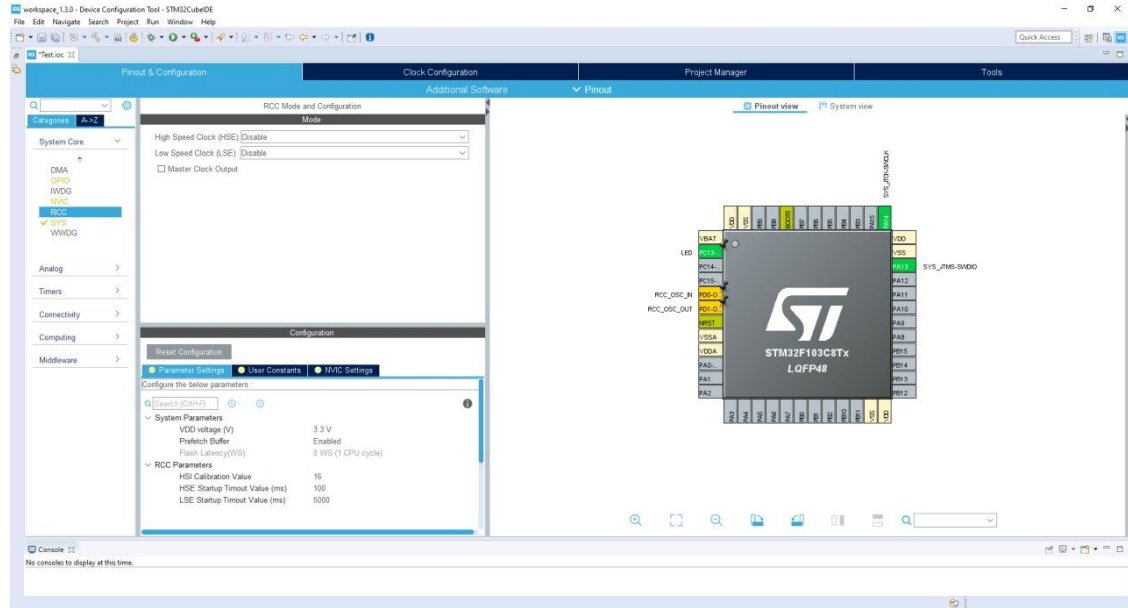


Overall Schematic



Software STM32CubeIDE

- It is the development IDE tool we are using for programming the STM32F7.
- It does the entire bootup sequence for us and also generates initialization code for all of the peripherals
- All of this is loaded into the STM's flash memory



Official API and drivers

BMP3 sensor API

Introduction

This package contains the Bosch Sensortec's BMP3 pressure sensor driver (sensor API)

The sensor driver package includes bmp3.h, bmp3.c and bmp3_defs.h files

Integration details

- Integrate bmp3.h, bmp3_defs.h and bmp3.c file in to your project.
- Include the bmp3.h file in your code like below.

```
#include "bmp3.h"
```

SH2 Sensorhub driver for MCU application

The files in this repository provide application-level SH2 sensor hub functionality.

To use this code, an application developer will need to:

- Incorporate this code into a project.
- Provide platform-level functions, as specified in sh2_hal.h
- Develop application logic to call the functions in sh2.h

More complete instruction can be found in the User's Guide:

- [SH2 Library User's Guide](#)

An example project based on this driver can be found here:

- [sh2-demo-nucleo](#)

DMA and NVIC (and EXTI)

Because UART is very slow (blocking), it essentially slows down our flight loop, even if the CPU is running at 216 MHz.

That's why we used Direct Memory Access (DMA) to combat that.

It moves UART8 TX data from Memory to Peripheral and RX data from Peripheral to Memory without slowing down the flight.

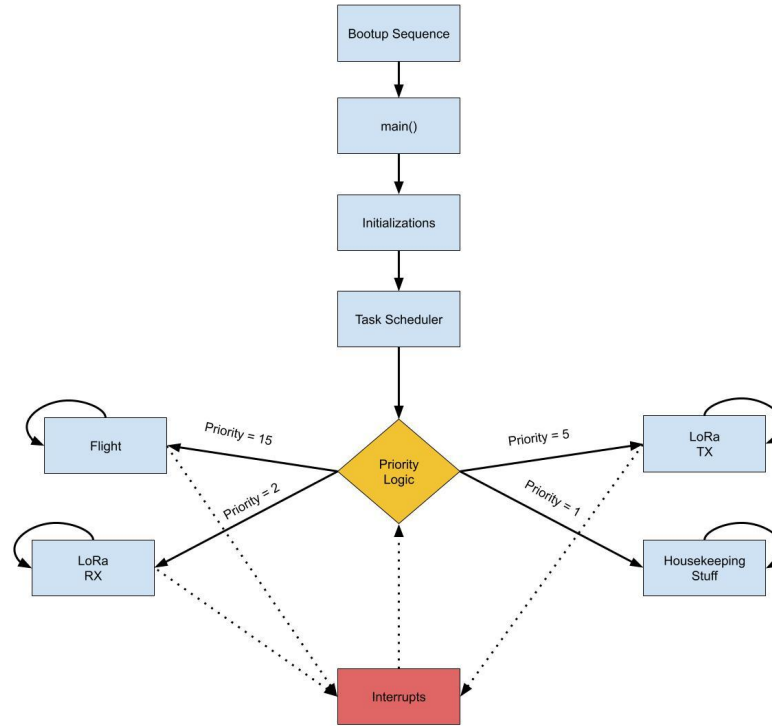
The Nested Vectored Interrupt Controller (NVIC) basically handles all our interrupts even the external ones (EXTI)

Real-time Operating System (FreeRTOS)

Instead of one giant loop, we used a Real-Time Operating System (RTOS) to help manage the various tasks.

FreeRTOS is a popular RTOS and is actually Middleware in CubeIDE using the “wrapper” called CMSIS-RTOS2 to link with the FreeRTOS kernel.





Tasks

There are four tasks (FlightTask, TelemTx, TelemRx, and Housekeeping) for the RTOS and three interrupt conditions.

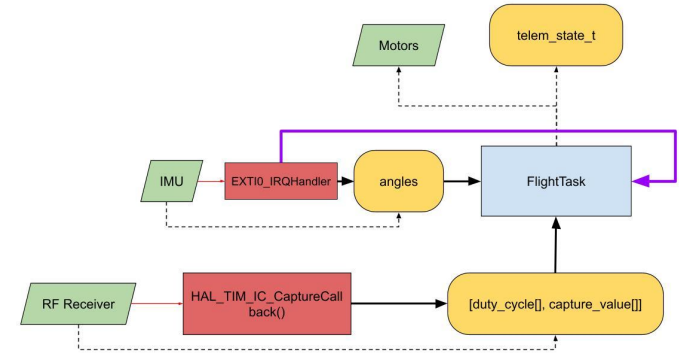
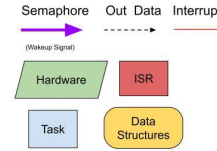
Realistically, running four at the same time with a single core is impossible.

BUT... we can give the illusion that it can by switching between tasks really **fast** and **efficiently** via a **task scheduler**.



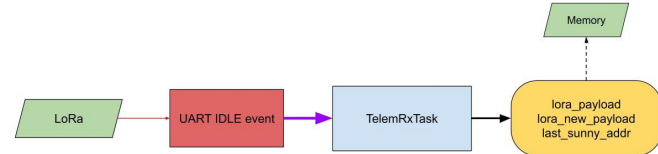
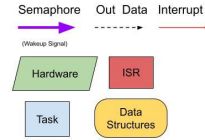
FlightTask (800 Hz - Interrupt-driven)

Purpose: Handles the core flight control loop, including reading IMU data, does PI, and updating motor PWMs.



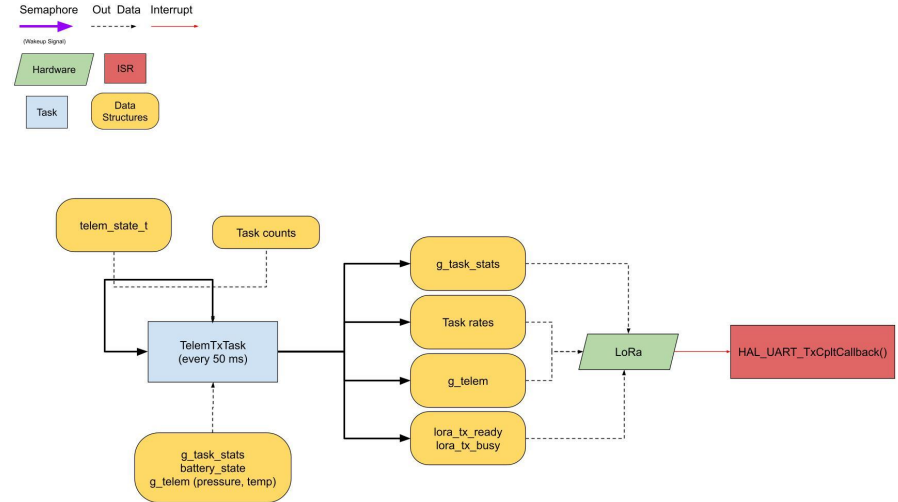
TelemRx (Interrupt-driven)

Purpose: Processes RX UART data from the LoRa, including handling "Sunny" pings, "Mari" commands.



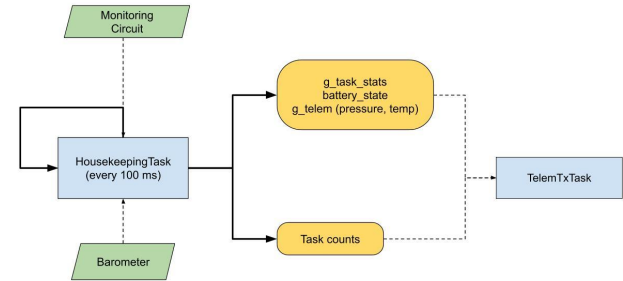
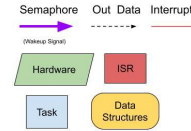
TelemTx (100 Hz)

Purpose: Manages the scheduled transmission of telemetry data (IMU, motor states, battery, and PID values) back to the ground station.



Housekeeping (10 Hz)

Purpose: It calculates how loops per second each task does (Hz), updates CPU usage statistics, reading battery voltage, and updating the barometer data.



Task Priority

To achieve this, task execution is based on priority and must also ensure that resources are allocated properly.

It uses a combination of polling and interrupts to achieve our results

High

FlightTask

TelemTxTask

Medium

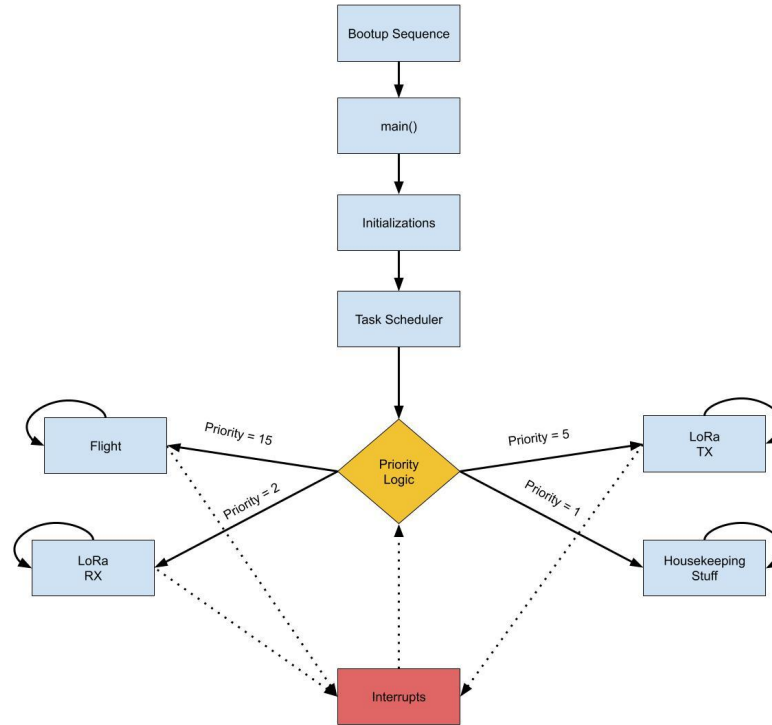
TelemRxTask

Low

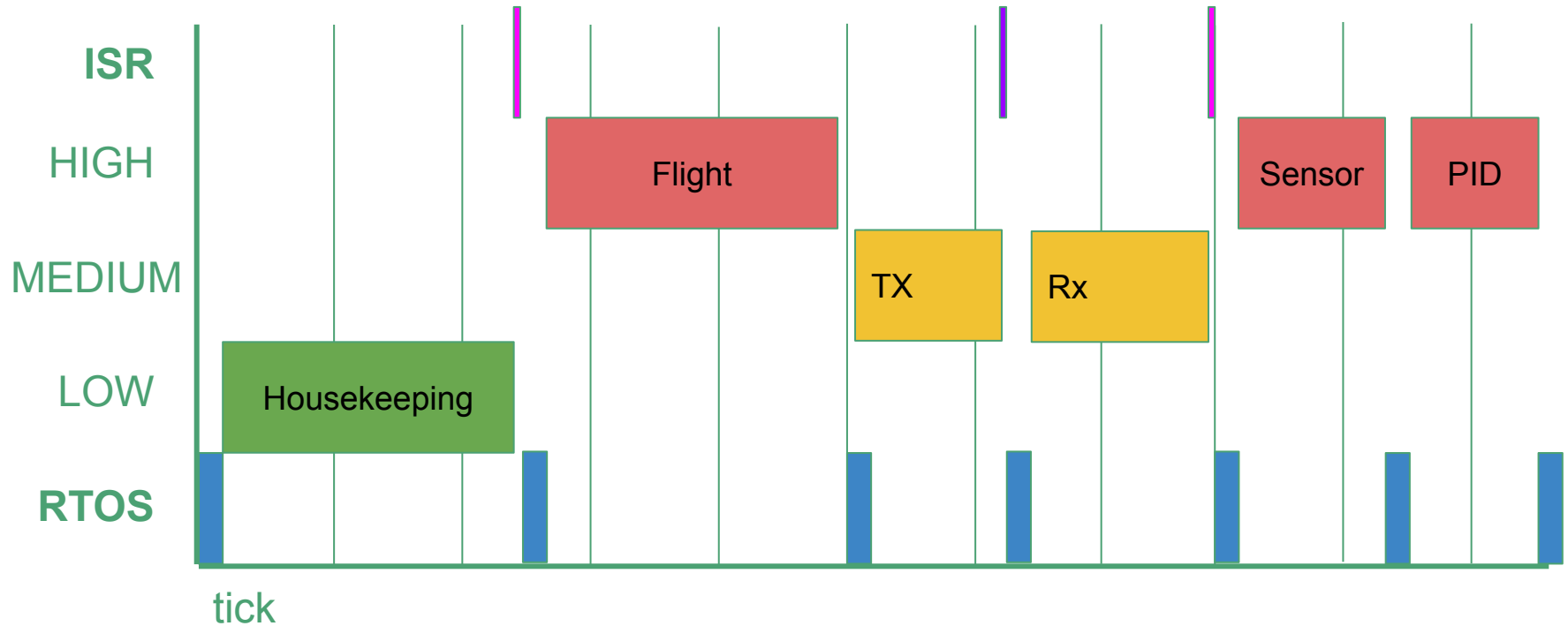
HousekeepingTask



Tasks for the MARI



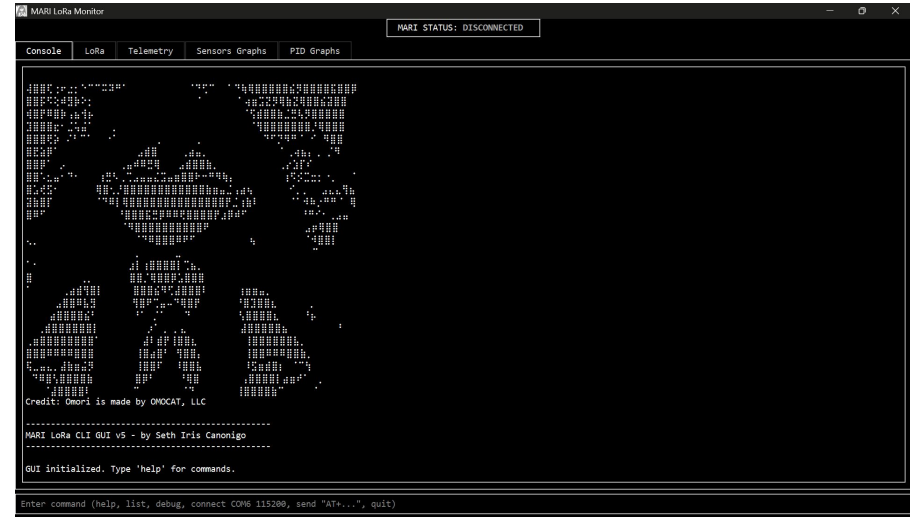
Task Scheduler



MARI LoRa Monitor

To keep track of the data and parse the data we get from the LoRa telemetry.

It functions as serial monitor, graphing tool,



MARI STATUS: **CONNECTED**

Console

LoRa

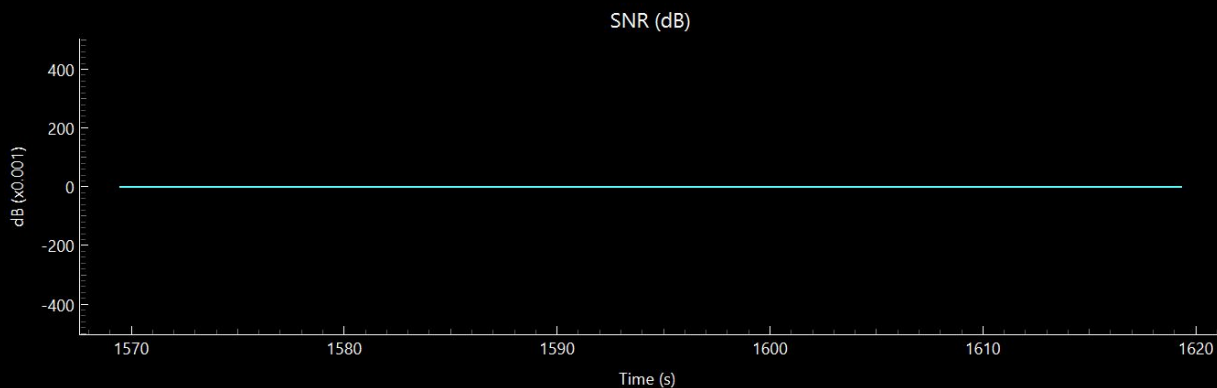
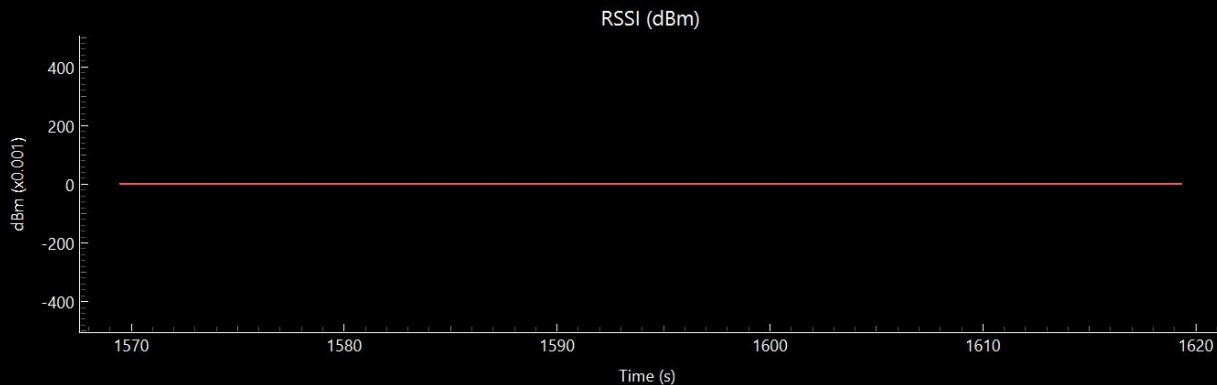
Telemetry

Sensors Graphs

PID Graphs

Fine-tuning

Metric	
Debug Mode	OFF
Display RX	ON
Serial Connection	CONNECTED
Port	COM3
Baud	115200
Last RSSI (dBm)	0
RSSI Okay?	-
Last SNR (dB)	0
SNR Okay?	-
Link Quality	No Signal
	-
Last Data	+OK
Last UUID	RW:0
Last Type	RAW
Last Sequence	0
Last Payload	+OK
Last Length ...	3
MARI Target Addr	0
	-



Enter command (help, list, debug, connect COM6 115200, send "AT+...", quit)

Console

LoRa

Telemetry

Sensors Graphs

PID Graphs

Fine-tuning

Metric		Metric		Metric	
Pitch P	0.00	Pitch (deg)	0.00	Flight Rate (Hz)	0
Pitch I	0.00	Roll (deg)	0.00	Tele TX Rate (Hz)	0
Pitch D	0.00	Yaw (deg)	0.00	Tele RX Rate (Hz)	0
	-	Temperature (°C)	0.0	Housekeeping Rate...	0
Roll P	0.00	Pressure (hPa)	0.0		-
Roll I	0.00	Altitude (m)	0.00	Flight CPU %	0.0
Roll D	0.00	Batt Volt (V)	0.00	Tele TX CPU %	0.0
	-	Battery Status	NORMAL	Tele RX CPU %	0.0
Yaw P	0.00		-	Housekeeping CPU %	0.0
Yaw I	0.00		-	Idle CPU %	0.0
Yaw D	0.00	Motor NW	0.00		
	-	Motor NE	0.00		
Pitch PID (Total)	0.00	Motor SW	0.00		
Roll PID (Total)	0.00	Motor SE	0.00		
Yaw PID (Total)	0.00		-		
		dt (ms)	0.00		
		max dt (ms)	0.00		
			-		
		Time	-		

Enter command (help, list, debug, connect COM6 115200, send "AT+...", quit)

MARI STATUS: **CONNECTED**

Console

LoRa

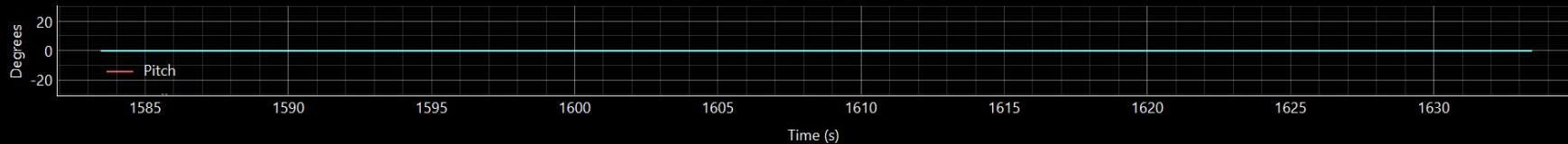
Telemetry

Sensors Graphs

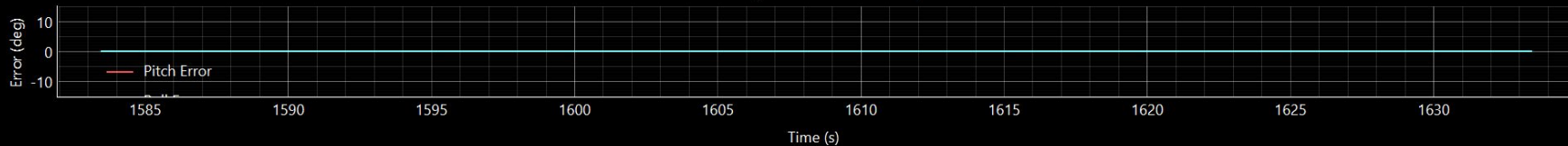
PID Graphs

Fine-tuning

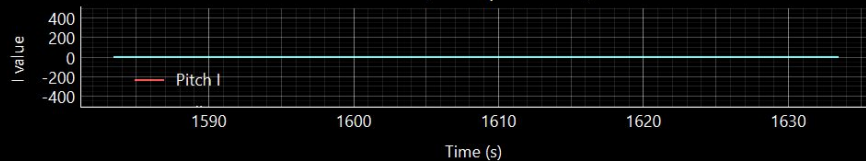
Attitude (solid) vs User Setpoint (dashed)



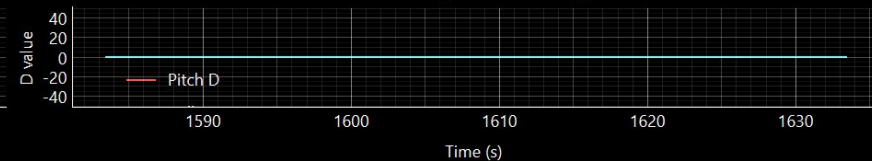
Error (oscillation indicator)



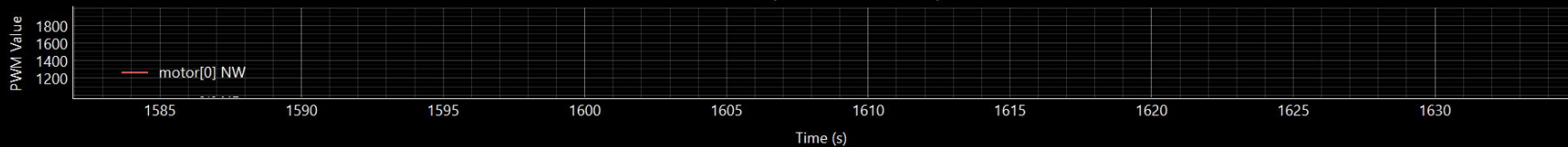
I terms (windup indicator)



D terms (noise indicator)



Motors (saturation indicator)

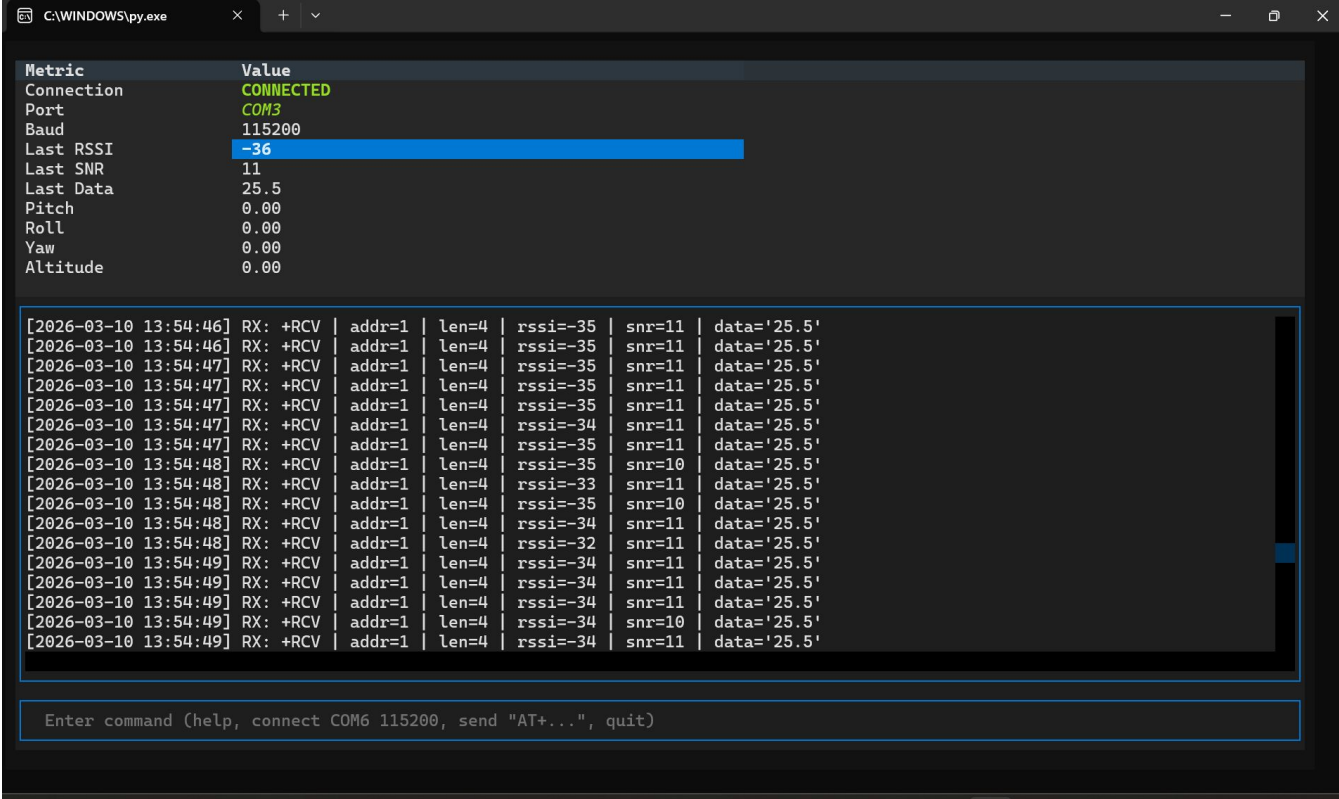


Enter command (help, list, debug, connect COM6 115200, send "AT+...", quit)

Version 1 and 2 (Plain terminal)

[illegible]

Version 3 (switched to Textual)



```
C:\WINDOWS\py.exe
```

Metric	Value
Connection	CONNECTED
Port	COM3
Baud	115200
Last RSSI	-36
Last SNR	11
Last Data	25.5
Pitch	0.00
Roll	0.00
Yaw	0.00
Altitude	0.00

[2026-03-10 13:54:46]	RX: +RCV	addr=1	len=4	rss=-35	snr=11	data='25.5'
[2026-03-10 13:54:46]	RX: +RCV	addr=1	len=4	rss=-35	snr=11	data='25.5'
[2026-03-10 13:54:47]	RX: +RCV	addr=1	len=4	rss=-35	snr=11	data='25.5'
[2026-03-10 13:54:47]	RX: +RCV	addr=1	len=4	rss=-35	snr=11	data='25.5'
[2026-03-10 13:54:47]	RX: +RCV	addr=1	len=4	rss=-35	snr=11	data='25.5'
[2026-03-10 13:54:47]	RX: +RCV	addr=1	len=4	rss=-34	snr=11	data='25.5'
[2026-03-10 13:54:47]	RX: +RCV	addr=1	len=4	rss=-35	snr=11	data='25.5'
[2026-03-10 13:54:48]	RX: +RCV	addr=1	len=4	rss=-35	snr=10	data='25.5'
[2026-03-10 13:54:48]	RX: +RCV	addr=1	len=4	rss=-33	snr=11	data='25.5'
[2026-03-10 13:54:48]	RX: +RCV	addr=1	len=4	rss=-35	snr=10	data='25.5'
[2026-03-10 13:54:48]	RX: +RCV	addr=1	len=4	rss=-34	snr=11	data='25.5'
[2026-03-10 13:54:48]	RX: +RCV	addr=1	len=4	rss=-32	snr=11	data='25.5'
[2026-03-10 13:54:49]	RX: +RCV	addr=1	len=4	rss=-34	snr=11	data='25.5'
[2026-03-10 13:54:49]	RX: +RCV	addr=1	len=4	rss=-34	snr=11	data='25.5'
[2026-03-10 13:54:49]	RX: +RCV	addr=1	len=4	rss=-34	snr=11	data='25.5'
[2026-03-10 13:54:49]	RX: +RCV	addr=1	len=4	rss=-34	snr=10	data='25.5'
[2026-03-10 13:54:49]	RX: +RCV	addr=1	len=4	rss=-34	snr=11	data='25.5'

Enter command (help, connect COM6 115200, send "AT+...", quit)

Version 4 (Remote Monitor)

```

LoRa Remote Monitor
Metric
Debug Mode OFF
Display RX ON
Connection DISCONNECTED
Port -
Baud 115200
Last RSSI (dBm) 0

##### .. #####+ .. +##-#####+ #####
+#####. .... ##### .. -#-#####+ ####
#####+ ..... +#####+ ..... +#####+ +##
##### .. #####+ ..... +#####+ ##
##### .. #####+ ..... -#####+ #
##### . -#####+ ..... #####
##### ##### ..... #####
#####+ -##### ..... #####

Credit: Omori is made by OMOCAT, LLC

-----
MARI LoRa CLI v4 Textual UI
Created by Seth Iris Canonigo
GitHub: siriscan
-----

Textual UI started. Type 'help' for commands.

Enter command (help, list, debug, connect COM6 115200, send "AT+...", quit)

```

LoRa **Remote Monitor**

Metric

Pitch (deg)	0.00
Roll (deg)	0.00
Yaw (deg)	0.00
Temperature (°C)	0.0
Pressure (hPa)	0.0
Altitude (m)	0.00
Battery Status	NORMAL
Throttle	0.00
Pitch Input	0.00
Roll Input	0.00
Yaw Input	0.00
Pitch PID	0.00
Roll PID	0.00
Yaw PID	0.00
Throttle Output	0.00
Pitch Output	0.00
Roll Output	0.00
Yaw Output	0.00
Motor NW	0.00
Motor NE	0.00
Motor SW	0.00
Motor SE	0.00
Time	00:00.0

Enter command (help, list, debug, connect COM6 115200, send "AT+...", quit)

Version 5 (switched to GUI, specifically PyQt6)



Graphs

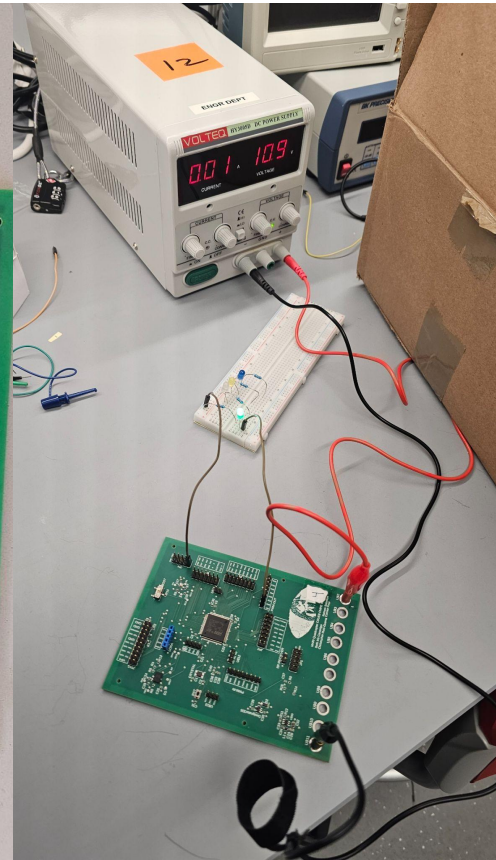
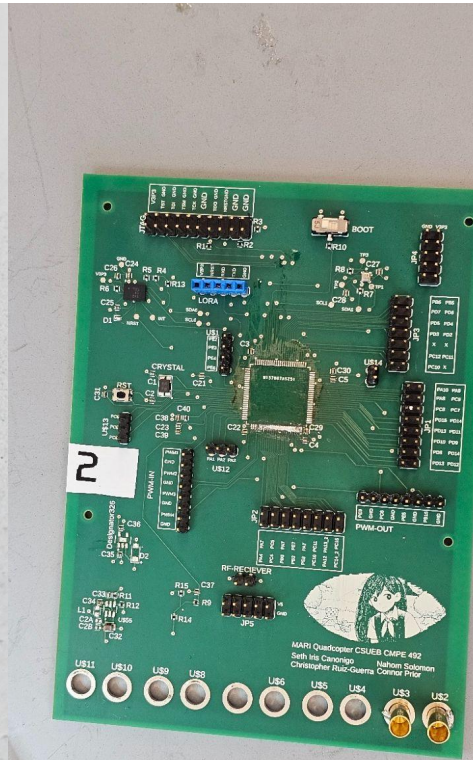
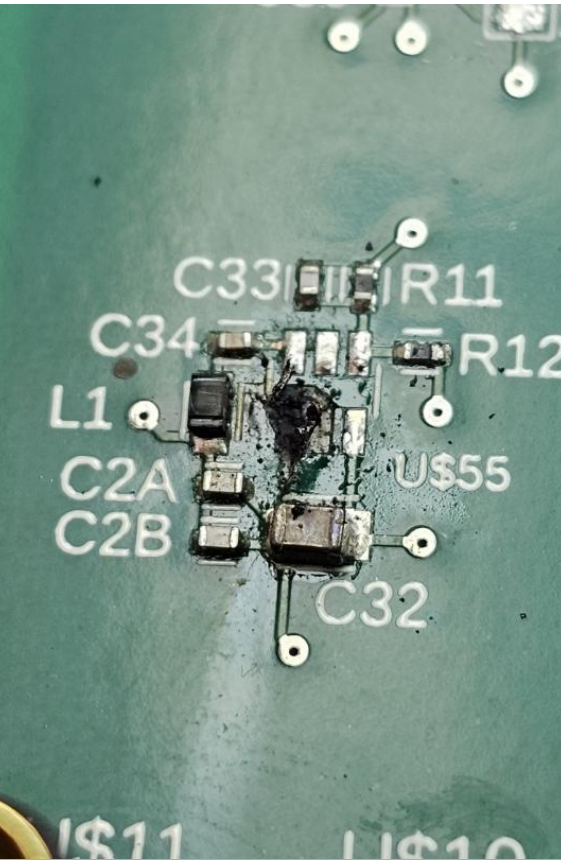


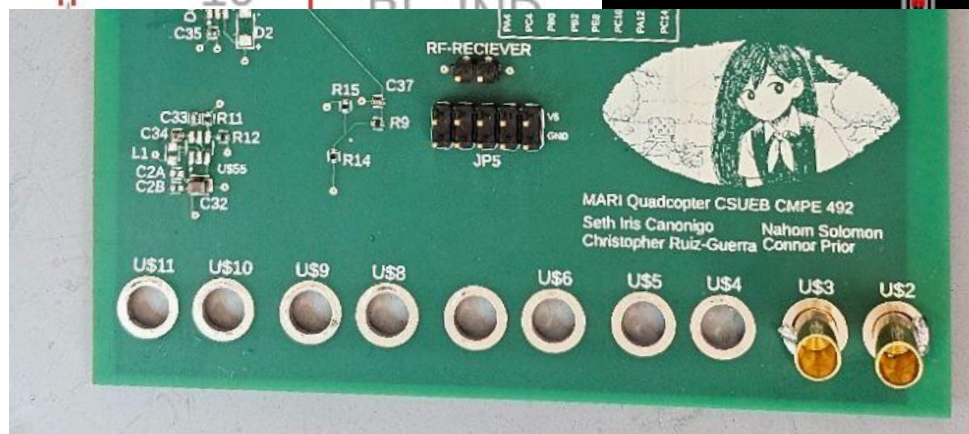
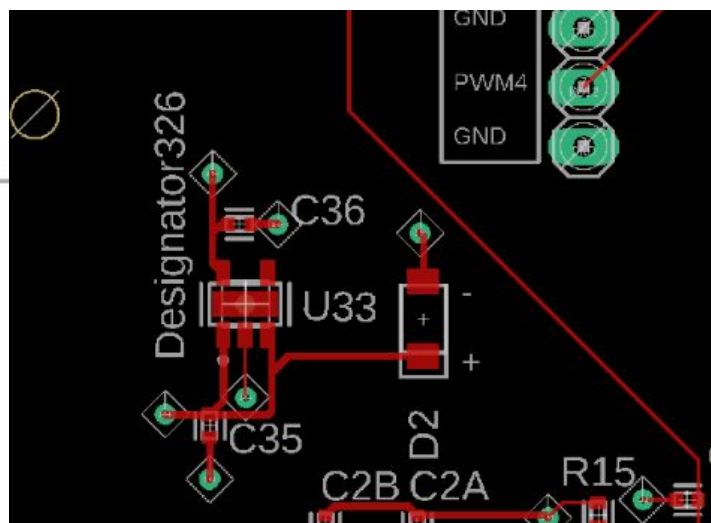
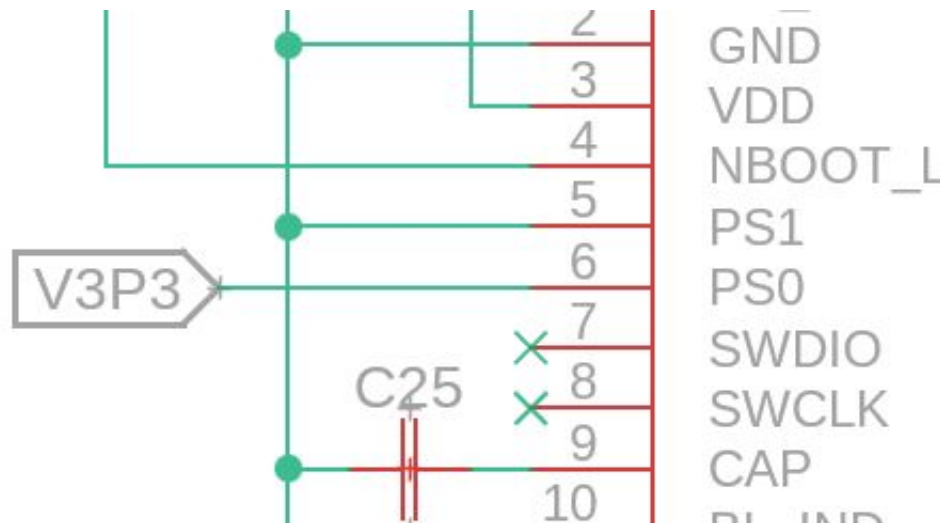
Difficulties: Needed to respin PCB

The first iteration of our PCB design was riddled with flaws:

- Zener diode was backwards
- Ultra-short burst, high-current draw causing a voltage spike
- The IMU was stuck in HID outputs
- LoRa was drawing too much current for the LDO
- LoRa had a lot of noise due to incorrect application configuration
- Missing VDD and VSS labels

PCB Failure Analysis: Fried BUCK & IMU





Causes: Transient Voltages

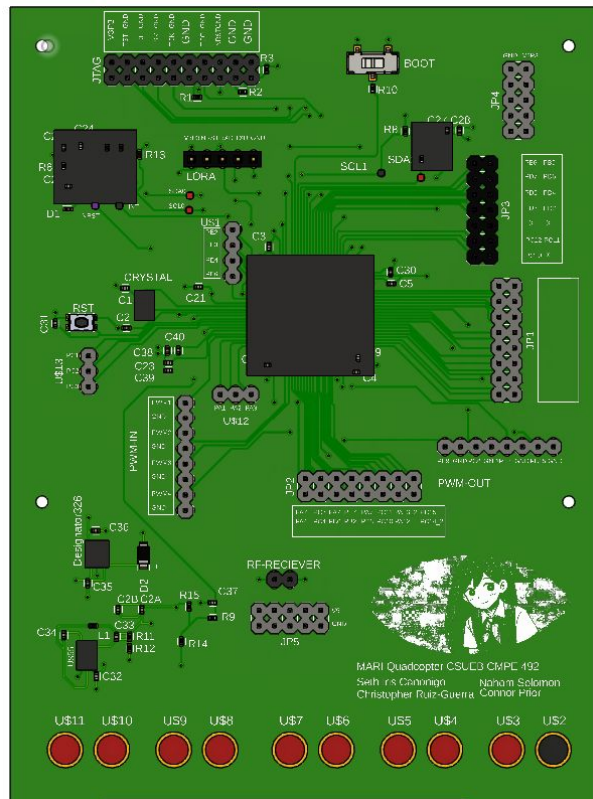
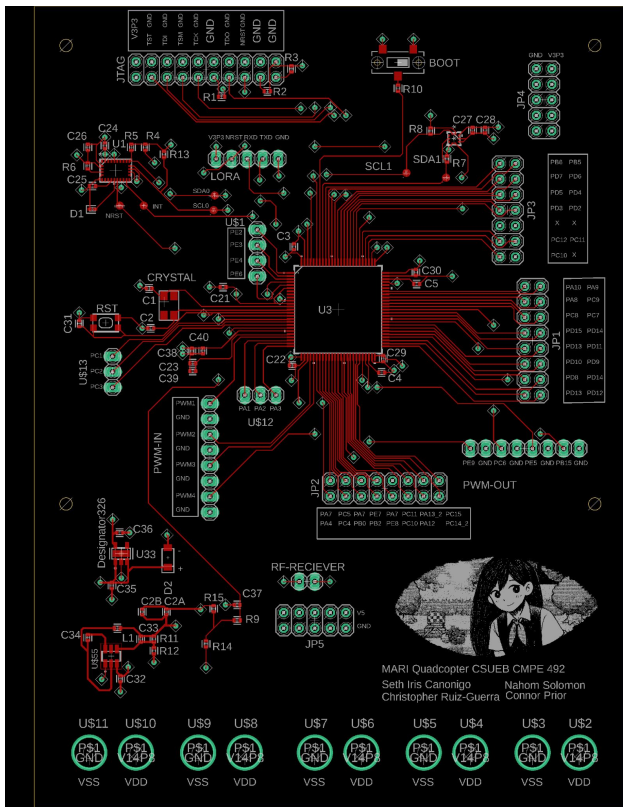


New PCB

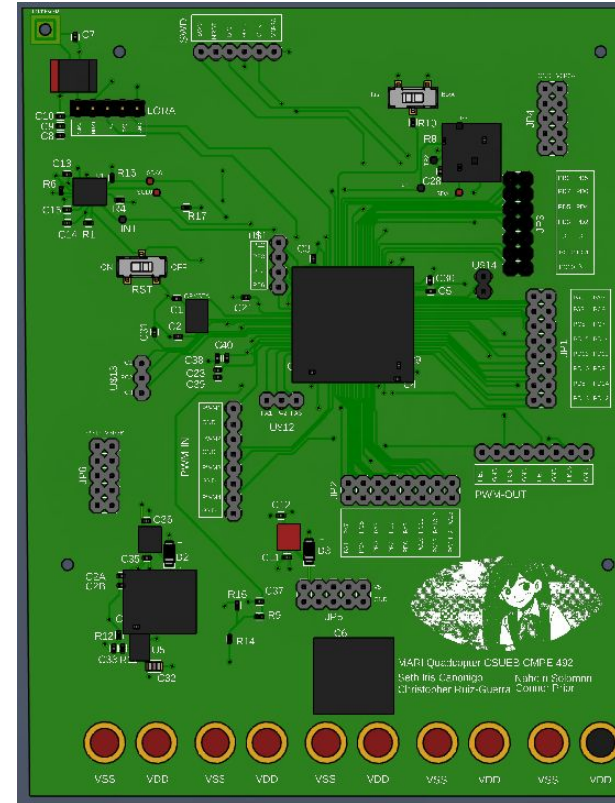
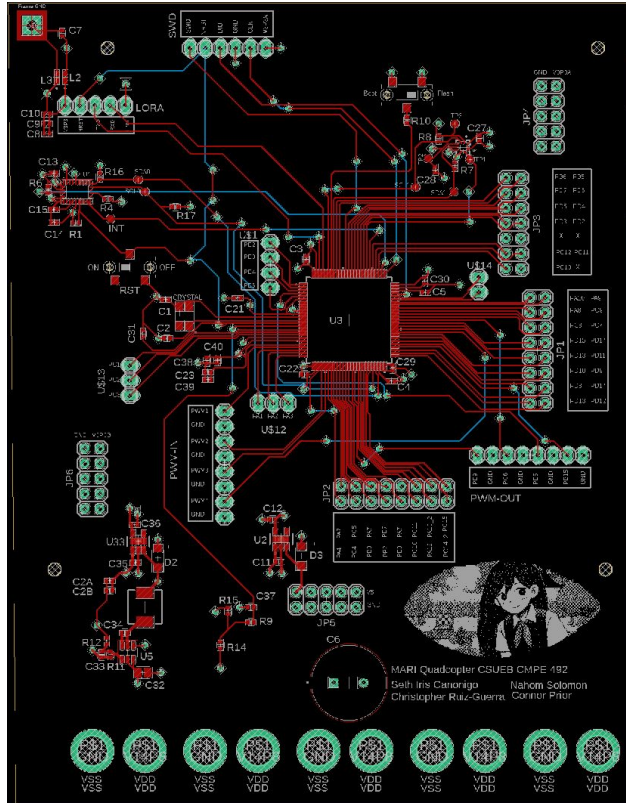
Changes

- Faster IMU (400 Hz vs 100 Hz)
- More power for LoRa (22 dBm vs 1 dBm)
- We have two LDOs for two separate 3.3 V planes
- New buck converter with inductor
- Big 1000 uF cap to dampen the spike
- Labeled everything

Old PCB



New PCB



Future Improvements

- Smaller PCB
- Better components with a higher sampling rate
- Included protocol with larger bandwidths and faster rates
- Better structured control system
- Additional domain for expansion ports
- Monitoring Circuit LEDs visible from distance
- Power switch
- Using SWV more often and knowing its features, like plotting

Conclusion

Quadcopter RC Drone with self-stabilization and altitude-lock

Operating System included to run our tasks almost concurrently.

The main goal of our drone is just to get it to fly stable in the air.

